四足机器人运动控制从零到精通教程

覆盖姿态、步态、速度、步态规划、步态切换、状态估计与工程实践

面向当前仓库的独立教程包 · 从基础理论到落地实现

学习主线:
站立 -> 姿态 -> 单腿 -> 慢走 -> 小跑 -> 切换 -> 高级控制 -> 仿真到实机

学习导航

建议先通读主线，再按章节动手复现，再回到仓库代码做映射。

- 10 分钟速览：从零到精通到底要经历什么
- 目录
- 第 1 章：先建立整体地图
  - 1.1 一次控制循环里到底发生了什么
  - 1.2 先记住这几组变量
  - 1.3 学习时最常见的断层在哪
  - 1.4 前六章真正的学习目标
  - 1.5 对照开源项目，你会发现它们其实很像
  - 1.6 本章源码对照阅读
- 第 2 章：四足机器人最基本的物理直觉
  - 2.1 先认识三个核心对象
  - 2.2 你必须非常熟的坐标系
  - 2.3 静态稳定和动态稳定
  - 2.4 支撑多边形和接触相
  - 2.5 运动学和动力学的区别
  - 2.6 初学者最容易混淆的三个量
  - 2.7 开源项目为什么都绕不开“ 足端层 ”
  - 2.8 本章源码对照阅读
- 第 3 章：姿态控制
  - 3.1 为什么姿态控制是第一课
  - 3.2 机身姿态如何映射到足端目标
  - 3.3 姿态控制的三种典型实现
  - 3.4 姿态控制的关键调参项
  - 3.5 姿态控制常见问题
  - 3.6 姿态控制实践建议
  - 3.7 先把一条腿讲透：从足端目标到关节角
  - 3.8 最小站立控制器怎么写
  - 3.9 开源项目通常怎样暴露姿态和站立命令
  - 3.10 本章源码对照阅读
- 第 4 章：步态基础
  - 4.1 步态的五个核心参数
  - 4.2 最常见的几种四足步态
  - 4.3 为什么 trot 这么常见

- 4.4 duty factor 的直觉
- 4.5 相位差是步态的灵魂
- 4.6 初学者构建步态的正确顺序
- 4.7 一个最小 gait scheduler 长什么样
- 4.8 开源项目里的 gait scheduler 共识
- 4.9 本章源码对照阅读
- 第 5 章：速度控制
- 5.1 速度命令如何进入步态控制
- 5.2 一个最常见的速度到着地点映射
- 5.3 速度控制中的三个容易忽略的东西
- 5.4 转向控制的直觉
- 5.5 速度控制常见问题
- 5.6 工程上怎么设计速度档位
- 5.7 速度命令怎样变成步长、步频和摆腿高度
- 5.8 开源项目里的速度接口为什么都很像
- 5.9 本章源码对照阅读
- 第 6 章：步态规划
- 6.1 步态规划通常包含四部分
- 6.2 摆腿轨迹设计
- 6.3 一个实用的摆腿高度经验
- 6.4 着地点规划不只是“往前放”
- 6.5 步长、步频、速度三者的关系
- 6.6 地形感知步态规划
- 6.7 步态规划的工程分层
- 6.8 支撑相与摆动相控制器如何配合
- 6.9 对照开源项目，步态规划里最值得借鉴的是什么
- 6.10 本章源码对照阅读
- 第 7 章：步态切换
- 7.1 步态切换为什么难
- 7.2 触发切换的常见条件
- 7.3 切换时最重要的三个原则
- 7.4 实用切换策略一：参数渐变
- 7.5 实用切换策略二：有限状态机
- 7.6 实用切换策略三：相位同步
- 7.7 切换中的迟滞和防抖

- 7.8 步态切换的安全守则
- 7.9 一条实用的切换路线
- 7.10 步态切换评估指标
- 7.11 一个 walk -> trot 切换落地流程
- 7.12 本章源码对照阅读
- 第 8 章：从位置控制到阻抗控制、力控制、WBC
- 8.1 位置控制
- 8.2 阻抗控制
- 8.3 力控制
- 8.4 Whole-Body Control
- 8.4.1 为什么 WBC 往往最后写成 QP
- 8.4.2 WBC 的输入、输出、决策变量
- 8.4.3 先看最小接触力 QP
- 8.4.4 再看最小 whole-body QP
- 8.4.5 约束、正则项、warm start 各自干什么
- 8.4.6 本仓库里的 WBC/QP 主链路
- 8.5 MPC 与 WBC 的典型关系
- 8.6 什么时候该升级控制方法
- 8.7 新手最需要避免的误区
- 8.8 本章源码对照阅读
- 第 9 章：状态估计与传感器
- 9.1 你至少需要哪些反馈
- 9.2 为什么 IMU 很关键
- 9.3 足接触检测有多重要
- 9.4 常见状态估计结构
- 9.5 状态估计中的工程重点
- 9.6 初学者可实现的最小状态估计链
- 9.7 本章源码对照阅读
- 第 10 章：完整控制架构
- 10.1 推荐的分层架构
- 10.2 为什么要分层
- 10.3 典型运行周期
- 10.4 实际工程中的模式管理
- 10.5 每一层到底输入什么、输出什么
- 10.6 本章源码对照阅读

- 第 11 章：从仿真到实机
  - 11.1 仿真里容易被忽略的东西
  - 11.2 正确的迁移顺序
  - 11.3 实机调试三原则
  - 11.4 从仿真迁移时最该检查的参数
  - 11.5 实机上线前的最后 12 项安全检查
  - 11.6 本章源码对照阅读
- 第 12 章：结合本仓库做系统实践
  - 12.1 推荐阅读顺序
  - 12.2 与教程概念对应的仓库模块
  - 12.3 一个很适合本仓库的学习顺序
  - 12.4 建议的仓库内练习题
  - 12.5 把本仓库和外部开源项目对照着读
- 第 13 章：调参与排障手册
  - 13.1 如果机器人站不稳
  - 13.2 如果机器人慢走还行，一加速就崩
  - 13.3 如果 trot 能跑，但切换失败
  - 13.4 如果落地冲击很重
  - 13.5 如果脚容易滑
  - 13.6 强烈推荐记录的日志
  - 13.7 最有用的可视化
- 第 14 章：从入门到精通的训练计划
  - 14.1 入门阶段：先把基本链路跑通
  - 14.2 初级阶段：会走
  - 14.3 中级阶段：会跑
  - 14.4 进阶阶段：会切换
  - 14.5 高级阶段：会做高级控制
  - 14.6 精通阶段：会做系统优化
- 附录 A：关键公式与直觉
  - A.1 机身速度到支撑足速度
  - A.2 简化着地点前推
  - A.3 一个常见摆腿高度函数
  - A.4 速度变化的工程约束
- 附录 B：步态切换伪代码
- 附录 C：从零到精通的最小实践清单

- 附录 D：术语表
- 附录 E：建议实验路线
  - E.1 零基础 7 步实验路线
  - E.2 每一步的验收标准
- 附录 F：实验记录模板
- 附录 G：真正做到“精通”前你要能回答的 20 个问题
- 附录 H：源码阅读导航
  - H.1 第一遍：只看主链路
  - H.2 第二遍：看高级低层控制
  - H.3 第三遍：看后端与状态同步
  - H.4 第四遍：看运行时与回归
- 附录 I：参数调优速查表
  - I.1 姿态类参数
  - I.2 步态类参数
  - I.3 摆腿类参数
  - I.4 切换类参数
- 附录 J：答辩 / 面试 / 评审常问问题
- 附录 K：最小控制器伪代码
- 附录 L：推导前必须会的最小数学准备
  - L.1 三角函数在四足控制里到底用来干什么
  - L.2 二维旋转矩阵的物理直觉
  - L.3 三维姿态为什么总是先讲 roll / pitch / yaw
  - L.4 小角度近似什么时候有用
  - L.5 为什么控制器里总有低通和斜坡
- 附录 M：单腿逆运动学从直觉到公式
  - M.1 先把问题缩成二维
  - M.2 先求膝关节角
  - M.3 再求髋关节角
  - M.4 为什么会出现“两支解”
  - M.5 从二维到三维怎么办
  - M.6 写进代码时最该防的 bug
  - M.7 最小 IK 伪代码
- 附录 N：姿态补偿、步态相位与落脚点公式
  - N.1 默认站姿为什么是整个系统的基准
  - N.2 机身姿态补偿的最小表达

- ◦ N.3 步态 phase 怎么变成 stance / swing
- ◦ N.4 为什么支撑足速度近似等于机身速度取反
- ◦ N.5 简化落脚点为什么会有 `0.5*T_stance*v_cmd`
- ◦ N.6 为什么还要加反馈修正项
- • 附录 O：最小状态估计公式
- ◦ O.1 为什么新手先学互补滤波就够了
- ◦ O.2 接触状态为什么能帮助状态估计
- ◦ O.3 最小速度估计直觉
- ◦ O.4 为什么滤波参数不可能“一把梭”
- • 附录 P：接触力 QP 从 wrench 到足端力
- ◦ P.1 为什么先单独学 contact QP
- ◦ P.2 wrench_map 怎么把各腿接触力合成机身 wrench
- ◦ P.3 摩擦锥、法向单边约束和投影
- ◦ P.4 anchor 与 share 两类正则项
- ◦ P.5 从足端力到关节力矩时为什么会有符号差异
- • 附录 Q：Whole-Body QP 从任务到关节力矩
- ◦ Q.1 为什么把 qdd 和 f 放进同一个变量
- ◦ Q.2 各任务怎样写成 J qdd ~= a*
- ◦ Q.3 为什么 torque consistency 可以写成正则项
- ◦ Q.4 解出来以后怎么恢复 tau
- ◦ Q.5 本仓库 whole_body_qp.py 的对照阅读
- ◦ Q.6 调参顺序与排障抓手
- • 附录 R：开源项目源码模式与参考代码骨架
- ◦ R.1 最小主循环骨架
- ◦ R.2 gait scheduler 的最小骨架
- ◦ R.3 速度命令整形骨架
- ◦ R.4 swing / stance 分发骨架
- ◦ R.5 足端目标到关节命令的骨架
- ◦ R.6 WBC 命令构造骨架
- ◦ R.7 你在教程里应该怎样标注“参考代码段”
- • 结语
- • 参考资料

四足机器人运动控制从零到精通教程

这是一份面向工程实践的四足机器人运动控制教程。目标不是只讲概念，而是把“能站起来、能走起来、能跑起来、能切换步态、能从仿真走向实机”的完整知识链条讲通。

适合对象：

- 刚接触四足机器人的学生或工程师
- 已能跑仿真，但对姿态、步态、速度、足端轨迹、切换逻辑理解不系统的人
- 已写过控制代码，但想把控制架构从“能跑”提升到“可解释、可调试、可扩展”的开发者

你将得到什么：

- 一条从零到精通的学习路线
- 一套四足机器人控制的统一心智模型
- 从基础理论到工程落地的关键公式、规则和伪代码
- 与当前仓库可直接对应的实践建议

版本说明：

- 当前为增强版教程，增加了学习地图、典型步态相位图、步态切换状态机、调参与回归闭环图
- 内容组织从“知识点堆叠”升级为“概念 -> 直觉 -> 方法 -> 调参 -> 落地”的教学路线
- 文末新增术语表、实验路线与实验记录模板，便于真正按教程动手
- 当前轮次继续补强了“公式推导加强版”，把最关键的数学关系补到能从概念走到代码

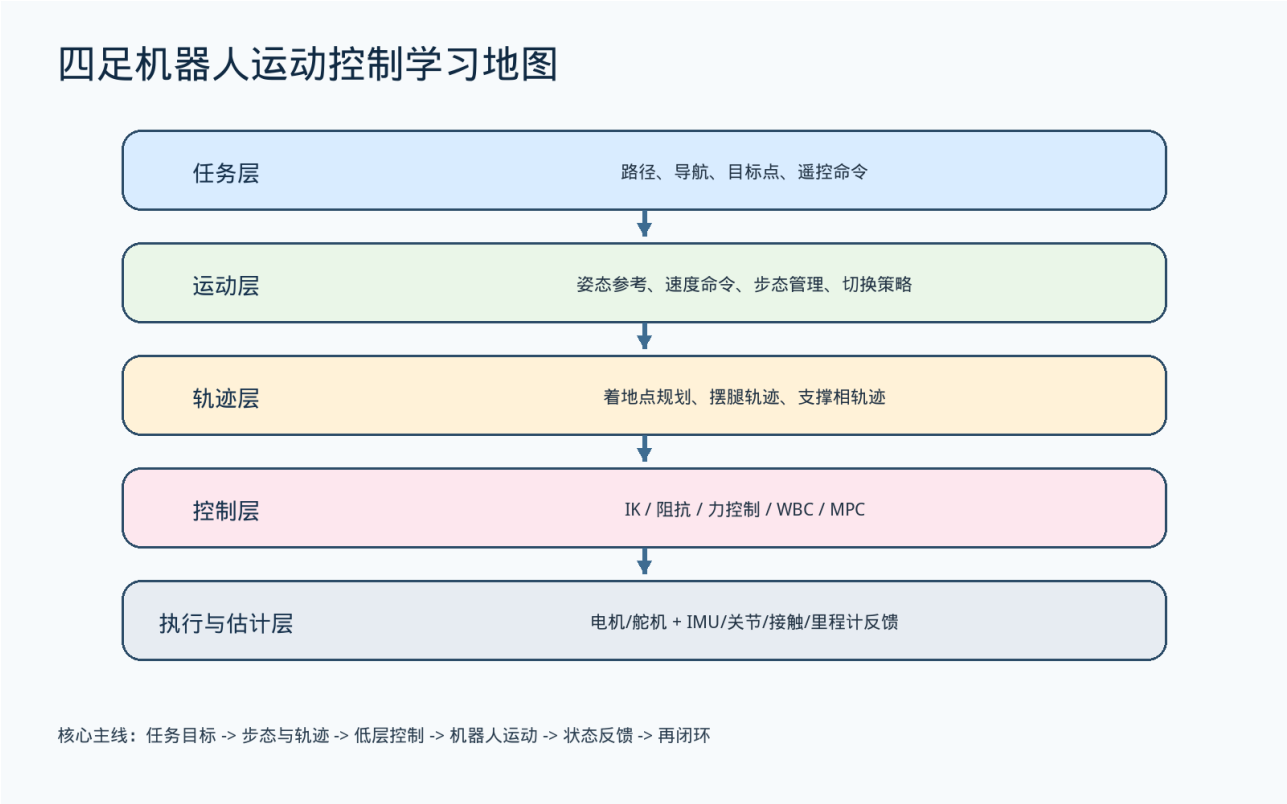
推荐使用方式：

- 第 1 遍：通读整份教程，先建立整体框架
- 第 2 遍：只抓姿态、步态、速度、步态规划、步态切换五大主线
- 第 3 遍：结合当前仓库模块对照实现
- 第 4 遍：按附录中的实验路线逐项做仿真与回归

建议先修知识：

- 刚体坐标变换、欧拉角、旋转矩阵
- PID、二阶系统、低通滤波
- 基本逆运动学与三角几何
- 一点点最优化和状态估计直觉

> 学习建议：不要把这份教程当“看完就会”的资料，而要当成“看一章、做一章、回归一章”的实战手册。



四足机器人运动控制学习地图

10 分钟速览：从零到精通到底要经历什么

如果你只有 10 分钟，请先建立下面这条升级路线：

- 级别 0：知道坐标系、腿编号、关节正负方向
- 级别 1：能稳定站立，能调高度和姿态
- 级别 2：能做单腿摆动和基本逆运动学
- 级别 3：能做静态 walk，理解相位和支撑多边形
- 级别 4：能做 trot，能响应 `vx / vy / wz`
- 级别 5：能做 walk/trot 切换，并理解迟滞和过渡态
- 级别 6：能做阻抗、接触反馈和状态估计闭环
- 级别 7：能理解 WBC / MPC 在整套系统中的位置
- 级别 8：能把仿真结果稳定迁移到实机
- 级别 9：能做结构清晰、可回归、可维护的运动控制系统

目录

- 第 1 章：先建立整体地图
- 第 2 章：四足机器人最基本的物理直觉
- 第 3 章：姿态控制
- 第 4 章：步态基础

- 第 5 章：速度控制
- 第 6 章：步态规划
- 第 7 章：步态切换
- 第 8 章：从位置控制到阻抗控制、力控制、WBC
- 第 9 章：状态估计与传感器
- 第 10 章：完整控制架构
- 第 11 章：从仿真到实机
- 第 12 章：结合本仓库做系统实践
- 第 13 章：调参与排障手册
- 第 14 章：从入门到精通的训练计划
- 附录：关键公式、伪代码、检查清单
- 附录：数学准备、IK 推导、步态公式、状态估计、WBC / QP 推导

第 1 章：先建立整体地图

很多人学四足机器人控制时会在细节里迷路：一会儿在调 PID，一会儿在调足端轨迹，一会儿又在看 MPC 论文，最后脑子里没有一条主线。

四足机器人运动控制，其实可以先压缩成下面这条链路：

```
任务目标
 -> 期望速度 / 期望姿态 / 期望路径
 -> 步态管理器
 -> 足端轨迹规划
 -> 逆运动学 / 阻抗 / 力控制 / WBC
 -> 电机或舵机命令
 -> 机器人实际运动
 -> 传感器反馈
 -> 状态估计
 -> 回到控制器
```

如果你只能记住一件事，请记住：

- 姿态控制回答的是“ 身体应该摆成什么样 ”
- 步态回答的是“ 哪条腿什么时候落地、什么时候抬腿 ”
- 速度控制回答的是“ 机器人整体以多快、多大转角速度运动 ”
- 步态规划回答的是“ 每只脚下一步该踩哪、走多高、何时落下 ”
- 步态切换回答的是“ 什么时候从站立切换到慢走、从慢走切换到小跑、再切到高速跑 ”

从学习顺序上，最稳妥的方法不是一上来学 WBC，而是按下面路径推进：

- 第一步：搞懂坐标系、腿部几何、身体姿态
- 第二步：让机器人稳定站立

- 第三步：让单腿抬起和落下可控
- 第四步：做最简单的静态步行
- 第五步：引入速度指令和连续步态
- 第六步：做 trot、pace、bound 等典型步态
- 第七步：做步态切换
- 第八步：再进入阻抗、力控制、WBC、MPC

从工程视角，成熟系统一般不是“一个算法统治一切”，而是分层：

- 上层：任务、路径、导航、速度命令
- 中层：步态、足端轨迹、着地点规划、姿态参考
- 下层：关节位置、速度、力矩、接触力控制
- 估计层：IMU、关节编码器、接触检测、里程计融合

1.1 一次控制循环里到底发生了什么

很多人第一次看四足控制代码时，会觉得文件很多、模块很多、状态很多，像一团线。

其实把一次控制循环压缩开来看，通常只有下面这几步：

```
读取传感器
 -> 估计机身姿态、速度、接触状态
 -> 读取用户命令或导航命令
 -> 生成姿态参考 / 速度参考 / 步态参考
 -> 计算每条腿当前是 stance 还是 swing
 -> 生成足端目标、接触力目标或关节目标
 -> 通过 IK / 阻抗 / 力控制 / WBC 输出电机命令
 -> 记录日志
 -> 进入下一拍
```

如果你以后看源码、看论文、看控制框图都记住这一条主线，就不会再轻易迷路。

1.2 先记住这几组变量

前六章里反复出现的，其实就是几类核心变量。

机身状态：

- `p_base`：机身位置
- `R_base`：机身姿态
- `v_base`：机身线速度
- `w_base`：机身角速度

关节状态：

- `q`：关节角
- `dq`：关节角速度

足端状态：

- `p_foot`：足端位置
- `v_foot`：足端速度
- `J_foot`：足端 Jacobian

步态状态：

- `phase`：全局相位
- `phase_i`：第 `i` 条腿局部相位
- `contact_i`：第 `i` 条腿当前接触状态

控制命令：

- `cmd = [vx, vy, wz]`
- `body_ref = [z, roll, pitch, yaw]`

你不需要一开始就把所有公式背下来，但必须知道：

- 机身层在管什么
- 足端层在管什么
- 关节层在管什么
- 它们之间靠什么变量连接

1.3 学习时最常见的断层在哪

前期最容易出现的“学了很多，但就是串不起来”，通常来自这几个断层：

- 只会看关节角，不会看足端目标
- 只会看姿态误差，不会看接触状态
- 只会看一帧结果，不会看完整周期
- 只会看控制输出，不会看控制输入是否合理

所以你在每一阶段都应该问四个问题：

- 输入是什么？
- 中间状态是什么？
- 输出是什么？
- 这一层出错时最先看哪个日志？

如果你把这四个问题养成习惯，后面学步态、速度、规划时会轻松很多。

1.4 前六章真正的学习目标

前六章的目标并不是让你“会一个完整商业级四足系统”，而是让你建立一套稳定的工程骨架：

- 第 1 章：知道整套链路有哪些层

- 第 2 章：知道机器人到底在和什么物理量打交道
- 第 3 章：把姿态控制链跑通
- 第 4 章：把步态时序搞清楚
- 第 5 章：把速度命令正确送入步态与足端层
- 第 6 章：把“脚什么时候抬、抬多高、落在哪”讲清楚

你可以把前六章看成一个递进关系：

```
建立地图
 -> 建立物理直觉
 -> 稳定站立
 -> 让腿按节律动起来
 -> 让机器人按速度命令运动
 -> 让脚真正踩到应该踩的地方
```

1.5 对照开源项目，你会发现它们其实很像

如果你去看几个典型开源项目，会发现前六章讲的骨架并不是“某一家仓库的写法”，而是很常见的工程共识。

以 Stanford Pupper 为例，它把控制主链明确拆成：

- gait scheduler
- stance controller
- swing controller
- inverse kinematics

这和本教程前六章的分层几乎一一对应：

- 第 4 章：谁在 stance、谁在 swing
- 第 5 章：速度命令如何进入各腿目标
- 第 6 章：摆腿轨迹和落脚点
- 第 3 章：最终如何落到足端与关节

再看 Unitree Guide，你会发现它给初学者暴露的是非常典型的有限状态机和底盘速度命令：

- Passive
- FixedStand
- Trotting
- 平移命令
- 转向命令

这说明一件很重要的事：成熟项目也不会一上来就把所有高级功能揉成一个黑盒，而是先把“站住”“切模式”“按速度走起来”这几件事单独做清楚。

再看 CHAMP 这类 ROS 四足框架，它会把很多关键步态参数直接作为配置暴露出来，例如：

- 最大前进 / 侧移 / 转向速度
- stance duration
- leg swing height
- walking height
- CoM 偏移

这也再次说明：前六章真正重要的不是堆数学，而是先把这些基础参数和链路建立成一个稳定、可调、可观测的系统。

1.6 本章源码对照阅读

如果你想把“整体地图”从概念变成读代码路线，最推荐按下面顺序：

- `Stanford Pupper Controller Description`
- 先只抓四个词：`gait scheduler`、`stance controller`、`swing controller`、`IK`
- 目的不是记细节，而是先确认一个最小四足控制链通常怎样分层
- `StanfordQuadruped`
- 再回到仓库主页和入口，确认一个整机项目通常怎样把控制、硬件接口和运行入口串起来
- `Unitree Guide`
- 再看 mode、速度命令、实验入口怎样拆开
- 本地已下载的 Gitee 项目
- `refcode/py_apple_quadruped_robot/README.md`
- `refcode/tinymal_b_quadruped_robot/README.md`

建议你边读边回答四个问题：

- 主循环入口在哪里？
- 速度命令在哪一层进入？
- 步态层和控制层是否明确分开？
- 整机项目是否先把“能运行”与“能优化”分成两个阶段？

如果你读完这些项目后，脑子里仍然没有下面这条链：

输入 -> 模式 -> 步态 -> 足端 -> 关节 -> 执行器 -> 反馈

那就说明源码还没读到点子上。

第 2 章：四足机器人最基本的物理直觉

2.1 先认识三个核心对象

- 机身 Base：机器人的主体，通常关注位置、高度、滚转、俯仰、偏航
- 腿 Leg：每条腿由若干关节组成，一般至少有外展、髋、膝

- 足端 Foot：真正与地面接触的点，是步态和支撑分析的核心

再往下走一步，你要知道三者分别承担什么“控制角色”：

- Base 是你真正想控制的整体运动结果
- Leg 是把整体目标分解到局部机构上的执行通道
- Foot 是机身和地面发生关系的接口

为什么四足控制总是绕不开足端？

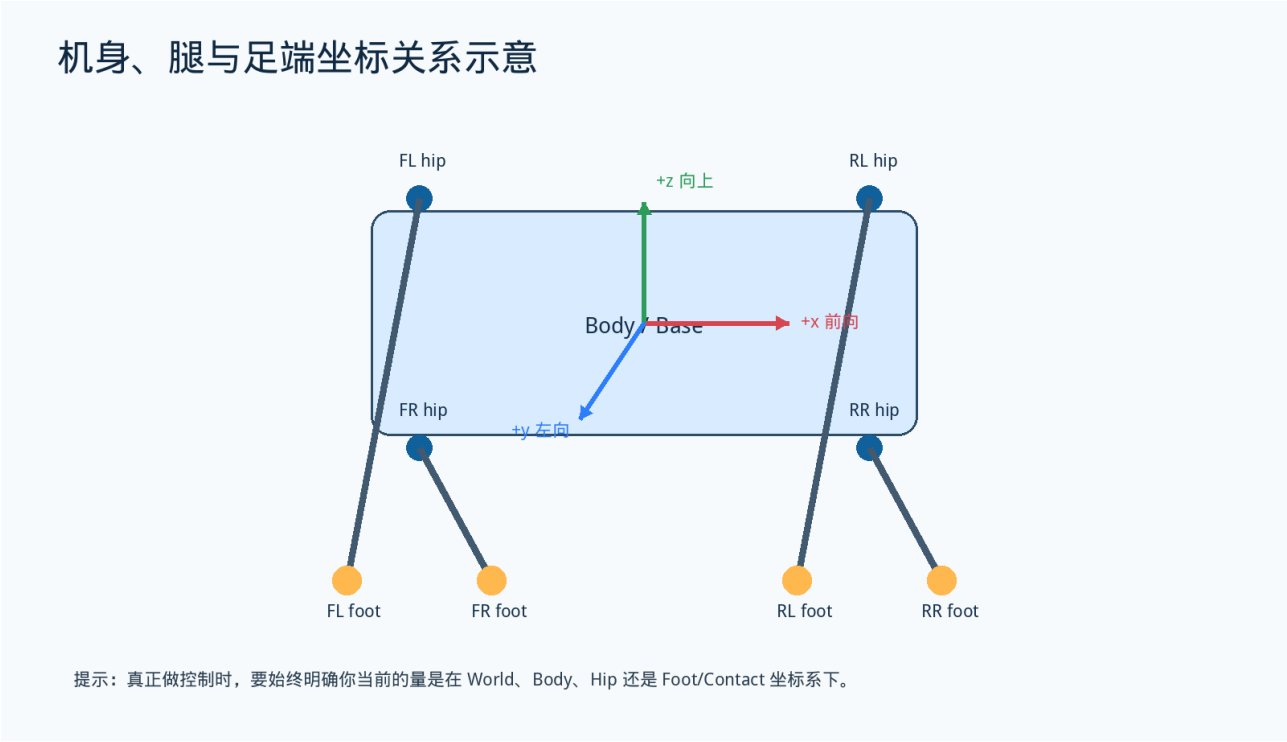
因为四足机器人不是靠轮子持续滚动前进，而是靠“接触点切换”前进。真正决定稳定性、推进力和落地冲击的，往往不是某个关节角本身，而是：

- 脚落在哪
- 脚什么时候落
- 脚落地时速度多大
- 脚着地后如何承载机身

2.2 你必须非常熟的坐标系

- 世界坐标系 World：看机器人在环境中的位置和方向
- 机身坐标系 Body：看机器人相对自己身体的姿态和足端位置
- 髋关节坐标系 Hip：做单腿逆运动学时最好用
- 足端接触坐标系 Contact：分析摩擦锥、法向力、切向力时常用

如果你不先统一坐标系，后面几乎所有 bug 都会变成“符号错了”“前后腿方向反了”“yaw 转换错了”“世界速度和机身速度混了”。



机身、腿与足端坐标关系示意

更具体一点，控制里最常见的转换链一般是：

```
World <-> Body <-> Hip <-> Foot
```

典型问题往往出在这里：

- 导航层给的是世界系速度，但步态层却直接当成机体系速度用
- 足端目标本来在机体系下定义，却被误当成世界系目标
- 左腿和右腿的 `y` 方向符号约定没统一

一个很实用的工程习惯是：

- 每个变量名字里直接带坐标系
- 每次打印日志时同时标明坐标系
- 每做一次旋转变换，都明确“是把向量转到新坐标系”，还是“把坐标轴本身旋转”

2.3 静态稳定和动态稳定

静态稳定：

- 只要机器人重心投影落在支撑多边形内，就有机会稳定站住
- 适合慢走、静态步态、姿态调试、最初 demo

动态稳定：

- 即使某一瞬间重心投影不在支撑多边形内，只要系统动量、足端着地点、接触力规划合理，也能稳定运动
- 小跑、奔跑、跳跃通常都属于动态稳定问题

工程上一个很重要的认知是：

- 站立和慢走，很多时候用几何 + 位置控制就够
- 跑起来、过障碍、切换快步态时，动力学建模的重要性会迅速提高

初学者可以先用一个最朴素的判据建立直觉：

```
如果 COM 投影在支撑多边形内，静态上更容易站住
如果 COM 投影跑到边界外，还想不倒，就必须靠速度、角动量、下一步落脚点和接触力救回来
```

这就是为什么：

- 站立控制更像“平衡一个姿态”
- 跑动控制更像“不断用下一步去接住身体”

2.4 支撑多边形和接触相

四足机器人控制的本质之一，是“控制接触”。

每条腿在任意时刻只有两种主要状态：

- stance：支撑相，脚在地上

- swing：摆动相，脚在空中

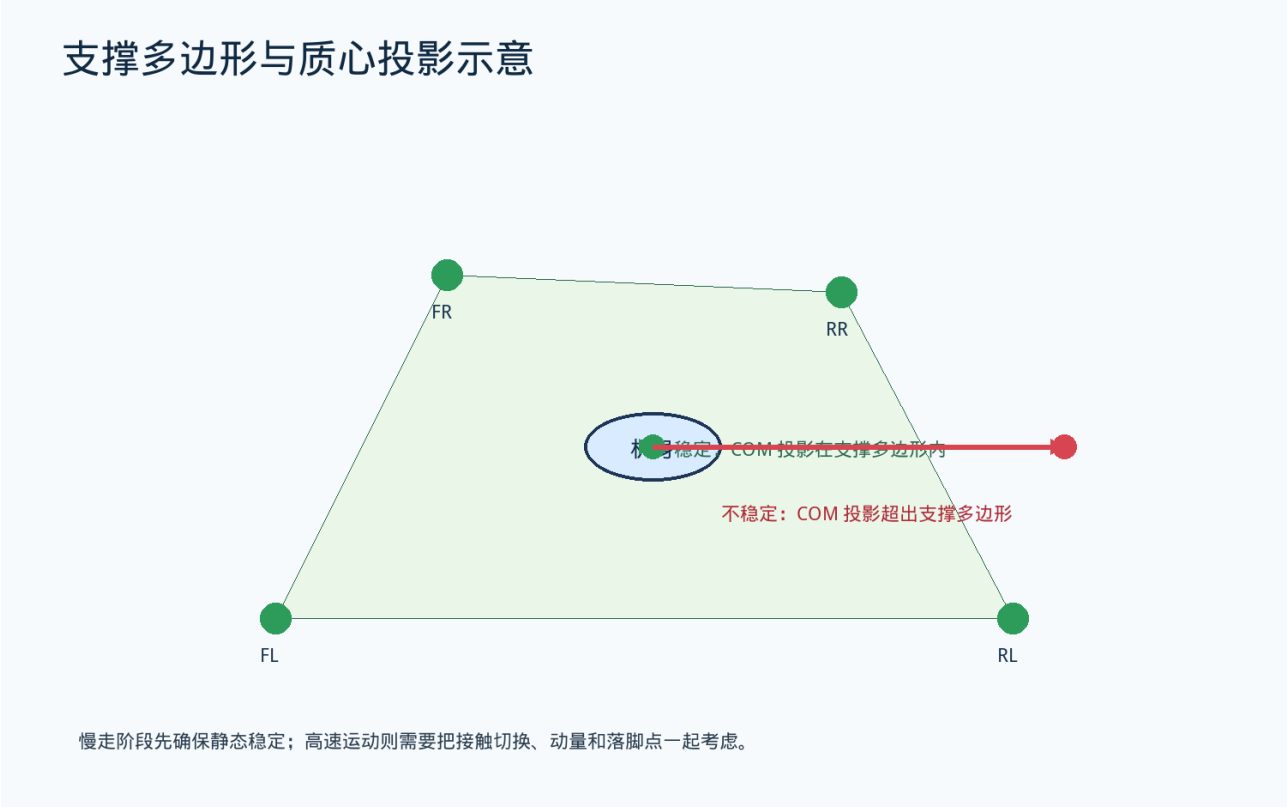
全机器人层面就是：

- 当前有哪些脚着地
- 这些脚形成怎样的支撑多边形
- 身体的重心、姿态、速度与该支撑条件是否匹配

因此，四足机器人是一个典型的混合系统：

- 连续动力学：身体和关节连续运动
- 离散事件：脚离地、落地、接触切换

步态，本质上就是接触序列的组织方式。



支撑多边形与质心投影示意

很多工程问题，其实都不是“控制公式错了”，而是“接触事件处理粗糙”：

- 落地判定抖动
- 刚落地就立刻被当成完全稳定支撑
- 已经滑了还被当成可靠接触

所以即使是最小系统，也建议你至少有：

- 接触阈值
- 接触防抖
- touchdown / liftoff 时间记录

- 支撑相与摆动相的过渡混合

2.5 运动学和动力学的区别

运动学关心：

- 足端在哪里
- 关节角度是多少
- 机身 roll/pitch/yaw 如何影响足端目标

动力学关心：

- 需要多大力矩
- 地面对脚的反作用力如何分配
- 机器人为什么会倒、会滑、会抖

新手最容易犯的错有两个：

- 只会运动学，不会接触与力分析，结果“位置都对，但一上地就崩”
- 一上来就啃复杂动力学，没有先把基本的姿态、步态、足端轨迹跑通

正确顺序是：

- 先用运动学把几何关系和参考轨迹打通
- 再用阻抗或力控制提升接触质量
- 再进入 WBC / MPC 这类更高级的全身控制

一个简单判断标准是：

- 如果你当前问题是“脚根本到不了目标位置”，先看运动学
- 如果你当前问题是“脚到了，但一碰地就弹、滑、抖”，先看动力学和接触
- 如果你当前问题是“多个目标同时满足不好”，再看优化控制

2.6 初学者最容易混淆的三个量

初学阶段最容易把下面三个量混成一团：

- 机身姿态：机身本身怎么转了
- 足端位置：脚尖在机身系或世界系的目标在哪里
- 关节角度：每个关节应该转多少

它们的关系是：

```
姿态与速度目标
 -> 足端目标
 -> 逆运动学
 -> 关节目标
```

一个非常常见的误区是：

- 看到机身前倾，就直接去改前腿髋关节角

这样做往往很难维护，因为：

- 你绕过了足端层
- 你很难解释“为什么这个角度有效”
- 你以后换腿长、换默认站姿、换机器人结构时几乎全部都要重调

更稳定的做法是：

- 先决定脚尖应该放在哪里
- 再让 IK 去算关节角
- 最后再考虑执行层是位置、阻抗还是 WBC

> 记住一句话：姿态、速度和步态最好优先在“机身与足端层”表达，而不是直接在“关节角层”硬改。

再把这句话翻译成调试顺序，就是：

```
先看机身参考对不对
 -> 再看足端目标对不对
 -> 再看 IK 输出的关节角对不对
 -> 最后看执行器是否跟上
```

如果你跳过中间层，后面几乎没法系统定位问题。

2.7 开源项目为什么都绕不开“足端层”

你如果去对照开源项目，会发现虽然代码风格不同，但很少有项目把上层命令直接写成“某个关节转多少度”。

更常见的做法是：

- 先定义机身目标
- 再定义足端目标
- 最后由 IK 或更高级控制器把它变成关节命令

原因很现实：

- 机器人结构会变，腿长会变，零位会变
- 但“脚该放哪里、机身该摆成什么样”这种表达更稳定
- 这样未来从位置控制升级到阻抗、WBC 时，也不需要把上层全部推翻

所以前六章里反复强调足端层，不是教学偏好，而是和大量开源实现一致的工程抽象。

2.8 本章源码对照阅读

本章最适合对照“机身、足端、关节到底怎么分层表达”。

推荐阅读顺序：

- `Stanford Pupper Operation Interface`
- 重点看它为什么把用户输入暴露为 body / velocity 语义，而不是直接暴露关节角
- `open-quadruped`
- 重点看 6D body pose 和 3 自由度腿 IK 是怎样放在同一条链路里的
- 本地 `refcode/py_apple_quadruped_robot/Py-Apple Dynamics V4.0 二次开发教程.pdf`
- 重点看机身、腿、足端如何被组织
- 本地 `refcode/tinymal_b_quadruped_robot/1 项目总体介绍Every one can make quadruped robot.pdf`
- 重点看整机结构和控制对象是怎么对应起来的

读这一章源码时，不要试图一口气看所有控制算法，只抓三件事：

- body 层变量长什么样
- foot 层变量长什么样
- joint 层变量长什么样

只要这三层看清了，后面再进步态、速度和 WBC 时就不会乱。

第 3 章：姿态控制

姿态控制解决的是机身的“ 站姿问题 ”。

对于四足机器人，最常见的姿态参考包括：

- 机身高度 z
- 滚转角 roll
- 俯仰角 pitch
- 偏航角 yaw
- 机身相对支撑面的平移偏置

3.1 为什么姿态控制是第一课

如果机器人连下面这些都做不好，后面步态再漂亮也没用：

- 高度站不稳
- 左右腿承重不均
- pitch 一变化就膝盖打死
- yaw 转一点就足端漂很远
- 抬腿时机身严重倾倒

所以第一阶段目标不是“ 会跑 ”，而是：

- 平地稳定站 30 秒
- 高度可调

- roll / pitch / yaw 小幅调节时姿态可跟踪
- 单腿轻抬时其余三腿能撑住机身

这四条验收标准很重要，因为它们分别验证了四件不同的事：

- 稳定站立：默认站姿、关节目标和低层控制基本正常
- 高度可调：垂向链路和腿长裕量合理
- 小角度姿态可跟踪：姿态误差到足端修正的映射正常
- 单腿轻抬仍能站住：支撑三角与重心管理有基本鲁棒性

3.2 机身姿态如何映射到足端目标

假设你已经有一个机身中心坐标系下的默认站立足端位置：

```
FL: (+x, +y, -h)
FR: (+x, -y, -h)
RL: (-x, +y, -h)
RR: (-x, -y, -h)
```

当机身 pitch 前倾时，从机身视角看：

- 前腿相对机会更“短”
- 后腿相对机会更“长”

当机身 roll 左倾时：

- 左腿需要更“短”
- 右腿需要更“长”

因此，姿态控制通常会先在机身坐标系中计算姿态变化引起的足端相对位移，再送入逆运动学。

最简单的做法是：

- 先定义默认足端位置
- 对机身姿态参考做旋转变换
- 计算每只脚新的期望位置
- 用逆运动学得到关节目标

如果用更工程化的表达，可以写成：

```
p_foot_des = p_nominal + delta_p_attitude + delta_p_body_offset
```

其中：

- `p_nominal`：默认站姿下每条腿的足端参考
- `delta_p_attitude`：roll / pitch / yaw 引起的补偿
- `delta_p_body_offset`：机身高度、前后左右偏置引起的补偿

在小角度下，`delta_p_attitude` 还常常可以近似理解成：

```
delta_p ≈ - omega_small x p_nominal
```

这个式子不要求你现在就熟练推导，但你要知道它的物理意义：

- 机身想往某个方向转
- 足端参考就要按相反方向做几何补偿

3.3 姿态控制的三种典型实现

- 几何补偿：直接由 roll/pitch/yaw 改足端目标
- 位置闭环：姿态误差进入 PID，再变成足端修正
- 力或阻抗闭环：姿态误差最终转成接触力或关节力矩

新手推荐路线：

- 第一版：几何补偿 + 关节位置控制
- 第二版：IMU 闭环的姿态 PID
- 第三版：足端阻抗或 WBC

为什么这个路线靠谱？

- 第一版先验证坐标、IK、默认站姿
- 第二版再让姿态误差真正闭环
- 第三版才去提升接触柔顺性和抗扰动能力

如果一上来就直接上“ 力控姿态 ”，你很容易分不清问题到底是：

- 姿态参考定义错了
- 足端几何映射错了
- IK 错了
- 接触力模型错了

3.4 姿态控制的关键调参项

- 高度基准值：过低容易蹭地，过高容易腿接近奇异位形
- roll/pitch 补偿增益：过小补不动，过大容易振荡
- IMU 低通滤波：太小噪声大，太大响应慢
- 姿态命令限幅：避免用户一下给太大的姿态参考
- 足端目标限幅：避免 IK 进入不可达区域

> 实战经验：站立姿态调试时，先把高度和 roll/pitch 稳住，再去碰 yaw；否则你很难判断问题到底来自姿态补偿、坐标变换还是转向耦合。

一个更细的调参顺序通常是：

```
默认站姿
 -> 身体高度
```

-> roll
-> pitch
-> 小范围 body x/y 偏置
-> yaw

这样你每次只引入一类新耦合，定位会清楚很多。

3.5 姿态控制常见问题

- 机器人站着发抖：通常是增益过大、传感器噪声、关节死区或足端目标突变
- 一改 pitch 就前腿打直：默认站姿不合理或 IK 可达域不够
- 转 yaw 时脚乱跑：世界系和机体系混用了
- 单腿抬起时身体掉下去：支撑腿姿态补偿太弱或重心偏置不合理

更建议你把现象和根因一一对应：

- 高频抖动：增益过大、足端目标不连续、IMU 噪声大
- 低频晃动：增益过小、滤波太重、重心偏置不合理
- 某一侧总压得更低：默认站姿左右不对称或腿零位没校准
- 一转向就崩：yaw 和 lateral 补偿耦合、左右腿符号错、机身系速度没转对

3.6 姿态控制实践建议

- 先锁 yaw，只做高度、roll、pitch
- 再加入小范围 yaw 调节
- 始终先在慢速、小幅度命令下调通
- 把姿态误差、目标足端位置、IK 解、关节限位同时打日志

如果你要做最小回归，建议固定下面四组测试：

- 高度从低到高缓慢扫描
- roll 做小幅正负阶跃
- pitch 做小幅正负阶跃
- 抬起任意一条腿并保持 1 到 2 秒

只要这四组测试长期稳定通过，姿态控制通常就有了一个比较可靠的基座。

3.7 先把一条腿讲透：从足端目标到关节角

如果你是零基础，建议先别同时盯四条腿。先把一条腿讲透，整个系统会一下清楚很多。

一条典型三自由度腿，常见有：

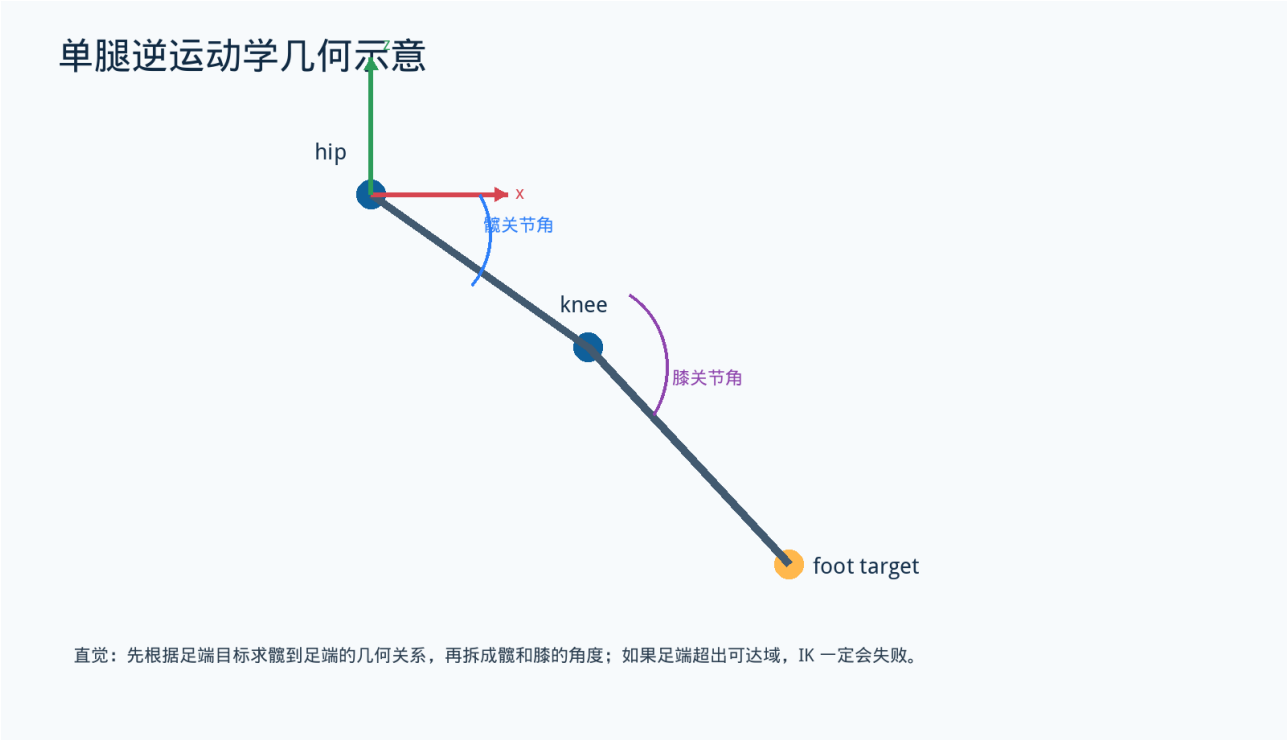
- 外展/内收关节
- 髋关节
- 膝关节

你真正需要解决的问题是：

- 给定一个足端目标 `(x, y, z)`
- 这条腿的三个关节应该分别转多少

理解 IK 最好的办法不是死记公式，而是先有几何直觉：

- 外展关节主要负责左右展开
- 髋关节和膝关节共同决定前后与上下的二连杆形状
- 当脚太远、太高或太靠近身体时，足端可能超出可达域



单腿逆运动学几何示意

做单腿 IK 时，你至少要检查四件事：

- 足端目标是否超出最大腿长
- 足端目标是否太靠近机身，导致膝部折叠异常
- 关节角是否超限
- 选取的膝部解是不是你想要的那支解

一个很实用的工程顺序是：

- 先固定 `y`，只在二维平面里理解髋膝二连杆
- 再加入 `y`，补上外展自由度
- 再加入机身姿态补偿，把默认站姿映射成四条腿的足端目标

3.8 最小站立控制器怎么写

很多教程会一下跳到“完整控制器”，但对初学者最关键的是先做一个最小站立控制器。

它的最小输入：

- 期望机身高度
- 期望 roll / pitch / yaw

它的最小输出：

- 四条腿的足端目标
- 或四条腿对应的关节目标

最小站立控制器的工作顺序：

1. 读取 IMU 与关节状态
2. 计算当前姿态误差
3. 用姿态误差修正四足默认站姿
4. 得到四条腿的足端目标
5. 通过 IK 求关节目标
6. 下发位置命令
7. 限幅、记录日志、循环

它不需要一开始就有：

- 路径规划
- 步态切换
- MPC
- 复杂状态估计

它只要能稳定把“姿态误差 -> 足端修正 -> 关节目标”这条链路跑通，你就已经跨过了最关键的门槛。

3.9 开源项目通常怎样暴露姿态和站立命令

这一步如果你去看 Stanford Pupper
的操作接口，会发现它并没有直接让用户去调“前左髋关节目标角”“后右膝关节目标角”。

更常见的用户入口是：

- 前后平移命令
- 左右平移命令
- 机身高度
- 机身 roll
- 机身 pitch
- 机身 yaw / turn

这正好说明前面那条原则为什么重要：

- 姿态控制最好在 body 层暴露
- 足端控制最好在 foot 层组织

- 关节层更像执行细节，而不是用户交互层

如果你现在就在设计自己的接口，建议前期至少把下面这些量显式做成输入：

```
body_ref = [height, roll, pitch, yaw]
cmd_vel  = [vx, vy, wz]
```

哪怕内部实现还只是位置控制，这个接口设计也能让后面升级顺很多。

3.10 本章源码对照阅读

如果你现在正在学“姿态控制到底该怎么进系统”，最推荐对照下面几份资料：

- `Stanford Pupper Operation Interface`
- 重点看 `height / roll / pitch / yaw` 和 `vx / vy / wz` 是怎样并列成为用户输入的
- `open-quadruped`
- 重点看 body pose、gait 与 IK 是怎样接起来的
- 本地 `refcode/py_apple_quadruped_robot/Py-Apple Controller 使用说明书 V4.0.pdf`
- 重点看站立、姿态、控制入口是怎样面向使用者暴露的
- 本地 `refcode/py_apple_quadruped_robot/开源菠萝四足机器人项目--上手指南（先看我）.pdf`
- 重点看一个整机项目是如何把“先站住”放在真正的第一步

读时请只盯这条链：

```
body_ref -> foot target -> IK -> joint target
```

如果你发现某个项目一上来就让你直接改很多关节角，而没有把 body / foot
两层表达清楚，那它更像是“能跑的 demo 入口”，不太适合作为姿态控制教学主线。

第 4 章：步态基础

步态不是“某条腿怎么摆”，而是“所有腿的支撑和摆动如何按时间组织”。

4.1 步态的五个核心参数

- gait period：步态周期
- phase offset：各腿之间的相位差
- duty factor：单腿支撑相占整个周期的比例
- stride length：一步的长度
- cadence / frequency：步频

理解步态时，最重要的不是背名称，而是会问：

- 哪些腿同时落地
- 哪些腿同时抬起
- 每条腿在地上的时间占比多少

- 当前速度区间适合什么接触节律

更进一步一点，最好把它们之间的关系也建立起来。

最常见的全局相位写法是：

```
global_phase = (t / T) % 1.0
leg_phase[i] = (global_phase + phase_offset[i]) % 1.0
```

然后再用 duty factor 判断当前腿处于：

```
if leg_phase[i] < duty_factor[i]:
    state = stance
else:
    state = swing
```

一旦你接受了这个表达，很多步态其实只是参数不同：

- `T` 变了：整体快慢变了
- `phase_offset` 变了：腿间配合关系变了
- `duty_factor` 变了：支撑时间长短变了

4.2 最常见的几种四足步态

- Fixed Stand：四脚着地，站立
- Static Walk：一次只抬一条腿，最稳但最慢
- Crawl：慢速连续步态，通常 duty factor 较高
- Trot：对角腿同步，是四足机器人里最常见的高速基础步态
- Pace：同侧腿同步，有些平台容易实现横向快速移动
- Bound：前腿一组、后腿一组，适合更高动态
- Gallop：更高速、存在更复杂的前后腿时序和可能的腾空相

如果从“学习难度”和“稳定性”角度排序，大致可以这么理解：

- Stand：最容易，主要验证姿态控制
- Static Walk / Crawl：最稳，主要验证接触时序和落脚点
- Trot：工程上最常用，主要验证动态支撑和摆腿节奏
- Pace：对横向稳定和 roll 控制更敏感
- Bound / Gallop：更高动态，对姿态、接触力和落地冲击要求明显更高

4.3 为什么 trot 这么常见

因为 trot 在工程上是一个很好的折中：

- 速度比静态步态高
- 相位关系简单
- 对角支撑在很多平台上较稳定

- 规划与控制实现相对直接

很多项目从头到尾，第一条真正跑通的动态步态就是 trot。

再说得更细一点，trot 常见还有三个非常现实的原因：

- 对角腿成对出现，pitch / roll 上比较容易形成平衡
- 相位关系简单，调度器和轨迹生成器都容易实现
- 大多数中小型四足平台在结构上对对角推进更“顺手”

4.4 duty factor 的直觉

- 大于 0.75：更偏慢走、静态
- 接近 0.5：更偏小跑
- 小于 0.5：通常意味着更动态、更激进，甚至存在腾空

如果你把 duty factor 调太小，但姿态控制、足端轨迹、接触力分配还没跟上，机器人通常就会：

- 滑
- 拍地
- 俯仰大
- 落地冲击重

原因其实很直白：

- duty factor 越小，单腿在地上的有效承载时间越短
- 相同速度下，支撑腿就必须在更短时间内完成推进和支撑
- 摆腿还要在更短时间内抬起、前送、落地

所以 duty factor 不是一个“只决定观感”的参数，而是直接改了：

- 接触重叠时间
- 单腿承载压力
- 摆腿时间预算
- 控制器的容错空间

4.5 相位差是步态的灵魂

如果四条腿相位完全一样，那就不是步态，而是一起蹲起。

典型 trot 的直觉是：

- FL 和 RR 同相
- FR 和 RL 同相
- 两组之间差半个周期

你可以把相位差理解成“谁跟谁一起承担身体”。

举个最直接的例子：

- 如果 FL 和 RR 同相，它们会形成一组对角支撑
- 如果 FL 和 FR 同相，则更偏向前后分组或横向摆动特性

所以相位差不仅决定“看起来像什么步态”，还决定：

- 支撑多边形如何随时间变化
- 机身摇摆主要出现在 roll 还是 pitch
- 姿态控制和速度控制该重点补偿哪个方向

步态调试时，一定要画出相位图，而不是只看最终动作。

```
时间 ->
FL: stance ---- swing ---- stance ----
FR: swing ---- stance ---- swing -----
RL: swing ---- stance ---- swing -----
RR: stance ---- swing ---- stance ----
```

4.6 初学者构建步态的正确顺序

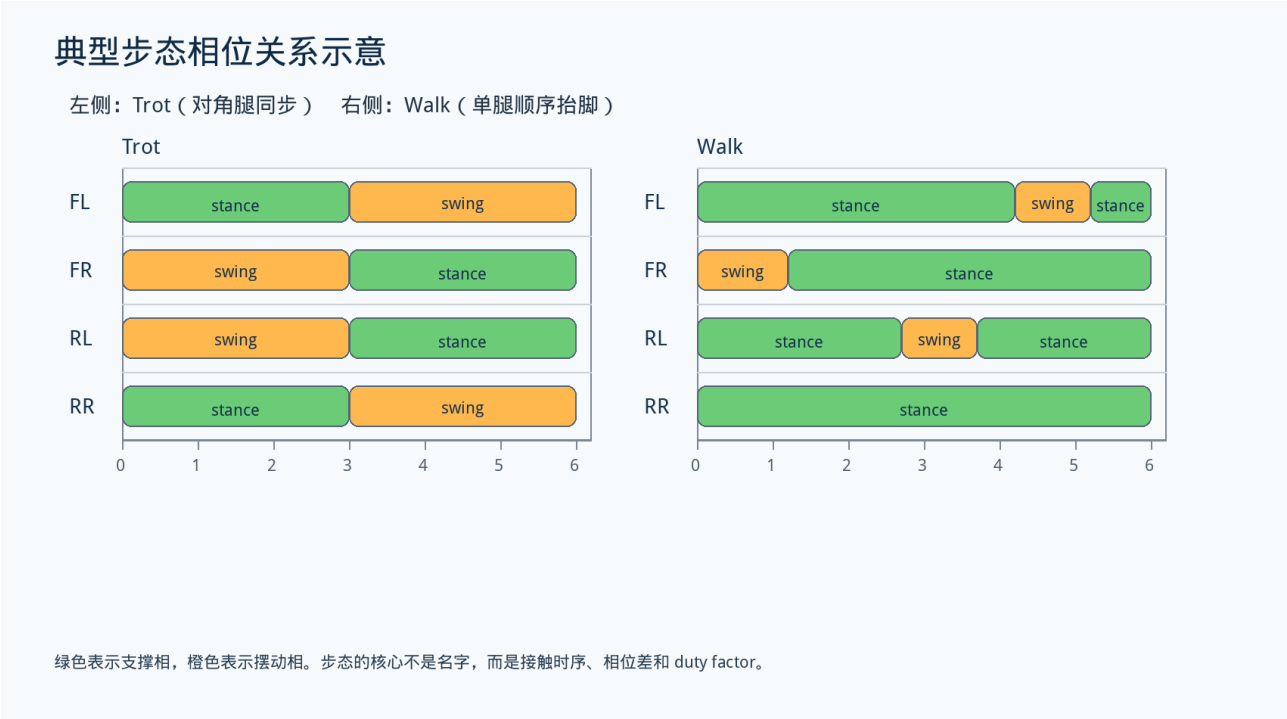
- 先做固定站立
- 再做单腿抬腿与落腿
- 再做四腿顺序慢走
- 再做对角腿 trot
- 最后再做 pace / bound / gallop

如果你一上来就做 gallop，绝大部分时间其实都在调姿态和落地冲击。

更细一点的工程顺序可以是：

```
先固定周期 T
 -> 再固定相位关系
 -> 再调 duty factor
 -> 再接入速度命令
 -> 再增加摆腿高度和落脚点前推
```

这样每次只新增一种复杂性，不会把“时序问题”和“轨迹问题”混在一起。



典型步态相位关系示意

4.7 一个最小 gait scheduler 长什么样

对新手来说，`gait scheduler`
最重要的作用不是“很智能”，而是“明确告诉你：当前每条腿到底是在支撑还是摆动”。

最小 gait scheduler 至少维护：

- 当前步态周期 `T`
- 当前时间相位 `phase`
- 每条腿的相位偏移 `phase_offset`
- 每条腿的 duty factor

判断逻辑通常是：

```
leg_phase = (global_phase + phase_offset[leg]) % 1.0
if leg_phase < duty_factor:
    leg_state = stance
else:
    leg_state = swing
```

这就是最小的步态调度器雏形。

一开始你不需要它很复杂，但你必须做到：

- 给定同样输入，它能稳定产生同样的时序
- 你能把每条腿当前状态打出来
- 你能把 phase 曲线画出来

只要这一层清楚，后面的 stance/swing 控制就不再是黑盒。

一个更像工程代码的最小版本，通常还会顺手维护：

- 每条腿当前相位
- 每条腿上次 touchdown 时间
- 每条腿上次 liftoff 时间
- 当前步态模式
- 是否处于步态切换中的过渡态

这样做的意义不是“让 scheduler 很高级”，而是让它足够可观测：

- 你能看到某条腿为什么切到 swing
- 你能看到切换是不是比计划提前或滞后
- 你能在后面接入接触检测，对计划时序做校正

4.8 开源项目里的 gait scheduler 共识

对照 Stanford Pupper、Unitree Guide、CHAMP 这类项目，你会发现步态层常有三种非常一致的做法：

- 用有限状态机区分 Stand / Walk / Trot 等模式
- 在模式内部再用 gait scheduler 生成每条腿的相位
- 用显式参数控制周期、支撑时长、摆腿高度和速度上限

这说明一个成熟步态系统通常不是“只有一个 gait 函数”，而是至少有两层：

- 模式层：我现在是站立、慢走还是小跑
- 时序层：当前模式下每条腿此刻是 stance 还是 swing

这一点对初学者特别重要，因为很多 bug 根本不是轨迹错了，而是：

- 进错模式
- 模式切换时序没对齐
- gait 参数在不同模式下复用了不合适的值

4.9 本章源码对照阅读

本章最适合拿来训练你“把模式层和相位层分开看”的能力。

推荐对照：

- `Stanford Pupper Controller Description`
- 看 `gait scheduler`、`stance controller`、`swing controller` 如何各司其职
- `Unitree Guide`
- 看 `Stand / FixedStand / Trotting` 这类模式怎样把 scheduler 放进 FSM
- `CHAMP`
- 看 `stance duration`、`leg swing height`、`walking height`、`CoM offset` 这些参数为什么要显式配置

建议你边读边画一张两层图：

模式层：Stand / Walk / Trot / Recovery
相位层：每条腿当前的 phase / stance / swing

只要你能把这两层分清，后面“步态切换失败”这类问题就会好定位很多。

第 5 章：速度控制

四足机器人速度控制最常见的输入不是“每个关节转多少”，而是：

- 期望前进速度 vx
- 期望侧移速度 vy
- 期望转向角速度 wz

也就是一个底盘式命令：

```
cmd = [vx, vy, wz]
```

5.1 速度命令如何进入步态控制

一种常见思路是：

- 步态调度器决定每条腿现在是 stance 还是 swing
- stance 腿根据机身期望速度，生成“相对机身向后滑”的足端速度
- swing 腿根据期望速度和步态周期，规划下一步落脚点

因此：

- 速度不是直接给关节
- 速度先变成“脚应该如何相对身体运动”
- 再经过 IK、阻抗或 WBC 下发到执行层

这一步里最容易被忽略的是“命令坐标系”。

很多系统上层给出的其实是世界系速度目标，而步态层更习惯用机体系速度：

```
v_body = R_world_to_body * v_world
```

如果这一步漏了，最典型的现象就是：

- 机器人转了个身，结果前进命令变成横移
- 导航层看起来给的是直行，步态层却像在侧移

5.2 一个最常见的速度到着地点映射

最简单的落脚点预测可以写成：

```
p_touchdown = p_hip_nominal
        + 0.5 * T_stance * v_cmd
        + k_yaw * yaw_rate_term
```

```
+ k_feedback * velocity_error
```

它的直觉是：

- 支撑相里脚会相对机身“向后扫”
- 摆动相结束时，要把脚提前放到下一步能支撑身体的位置
- 机器人越快，下一步落脚点通常要更往前
- 转弯时，左右脚、前后脚的落点会不一样

这个式子里每一项都值得你单独理解：

- `p_hip_nominal`：不动时，脚应该落在髋关节附近的哪个默认支撑点
- `0.5 * T_stance * v_cmd`：如果支撑相持续这么久，那脚就该提前半个支撑程
- `yaw_rate_term`：转弯时不同腿会围绕机身中心形成不同半径
- `velocity_error`：当真实速度跟不上时，落脚点要帮忙“接住身体”

所以落脚点规划不是单纯的几何前推，而是把：

- 速度命令
- 当前状态
- 步态周期
- 转向需求

统一折算成下一步足端应该出现的位置。

5.3 速度控制中的三个容易忽略的东西

- 限幅：用户命令不能直接原封不动用
- 斜坡：速度应有加速度限制，而不是瞬间跳变
- 机身系与世界系转换：导航层给的是世界方向，步态层常用机身方向

再往前多走一步，通常还建议加：

- deadband：很小的手柄抖动不应触发持续小碎步
- 低通：速度指令不要一拍一拍尖锐跳动
- 模式门控：站立模式、慢走模式、高速模式对最大允许速度不同

一个很实用的命令整形链可以写成：

```
raw_cmd
  -> deadband
  -> saturate
  -> rate limit
  -> low-pass
  -> gait / foothold planner
```

5.4 转向控制的直觉

如果机器人要原地左转：

- 左右腿的期望足端轨迹会围绕机身中心旋转
- 机身坐标系下，左右腿的切向速度方向相反

如果机器人要一边前进一边左转：

- 总体足端目标是“ 平移分量 + 旋转分量 ”的叠加

很多人第一次做 `wz` 时会发现：

- 直行能跑
- 原地转也还行
- 但一边前进一边转就开始乱

根因通常不是“ 转向很复杂 ”，而是你没有显式把两部分分开：

```
v_foot_des = v_translation + v_rotation
```

其中旋转项通常和足端相对机身中心的位置有关。

5.5 速度控制常见问题

- 命令一给就蹿：加速度限幅和频率限幅没做
- 低速抖动：步态频率过高、步长太小、摩擦模型不稳
- 高速拖腿：swing 轨迹高度不够或摆腿速度不够
- 转弯时翻车：yaw 控制和姿态支撑没匹配好

建议你以后看速度问题时，优先把它分成三类：

- 命令整形问题：速度输入太生硬
- 步态参数问题：步长、步频、摆腿高度不匹配
- 接触支撑问题：落脚点和姿态支撑没接上

5.6 工程上怎么设计速度档位

很实用的做法是给控制器定义几个工作区间：

- 低速区：walk / crawl
- 中速区：trot
- 高速区：fast trot / bound

这样后面做步态切换会简单很多，因为速度区间天然能成为步态选择条件。

进一步说，速度档位的意义不只是“ 切换一个名字 ”，而是同时切换一组参数：

- 步态类型
- 周期范围

- duty factor 范围
- 步长上限
- 摆腿高度上限
- 姿态控制权重

这会让你的控制器从“一个大杂烩参数表”，变成“不同速度区间对应不同工作点”。

5.7 速度命令怎样变成步长、步频和摆腿高度

很多新手知道输入是 `vx / vy / wz`，但不知道它最终如何影响步态参数。

一个实用思路是：

- 低速时：优先增大步长，步频变化小
- 中速时：步长和步频同时增长
- 高速时：步频继续增长，同时摆腿高度、姿态稳定权重、落脚点前推都会增强

你可以先做一个最简单的映射：

```
speed_mag = sqrt(vx^2 + vy^2)

stride   = clamp(k_stride * speed_mag, stride_min, stride_max)
freq     = clamp(k_freq   * speed_mag, freq_min, freq_max)
clearance= clamp(k_clear  * speed_mag, h_min, h_max)
```

然后再逐步加入：

- `wz` 对左右腿落脚点和横向速度的影响
- 姿态误差反馈
- 实际速度误差反馈

这样做的好处是非常循序渐进：

- 先跑通
- 再修正
- 再增强

如果想把这一步做得更稳，建议再加两条经验：

- 不要让步长和步频同时猛增，优先固定一个、缓慢拉另一个
- 速度下降时也要有滞后和斜坡，否则很容易在阈值附近频繁抖动

5.8 开源项目里的速度接口为什么都很像

不管是 Stanford Pupper 的手柄接口、Unitree Guide 的键盘接口，还是 CHAMP 暴露的速度参数，你都会反复看到同一组量：

```
vx / vy / wz
```

原因很简单：

- 对上层任务来说，这就是最自然的底盘语义
- 对步态层来说，这三维命令足以生成大多数基础足端目标
- 对下层控制来说，它又足够抽象，不依赖具体腿长和关节排布

所以如果你今天还在犹豫“上层到底该直接给关节角还是给速度命令”，开源项目已经给了非常强的工程答案：

- 上层更适合给底盘速度和机身姿态
- 下层再把它分解成足端与关节目标

5.9 本章源码对照阅读

本章最推荐你去对照“`vx / vy / wz` 是怎样真正进系统的”。

推荐对照顺序：

- `Stanford Pupper Operation Interface`
- 看手柄或操作输入如何最终收敛成底盘速度语义
- `Unitree Guide`
- 看键控和模式命令怎样进入运动层
- `CHAMP`
- 看最大前进、侧移、转向速度等参数怎样配置
- `DingoQuadruped`
- 看 driver / topic 侧怎样把上层命令接到机器人系统

读完后，建议你自己手画一条命令链：

```
raw_cmd
 -> deadband / saturate / rate limit
 -> gait mode
 -> foothold / swing / stance
 -> IK / low-level control
```

如果某个项目把这条链藏得很深，你就优先去找：

- 命令入口
- 命令整形
- 步态层接点
- 足端目标生成点

第 6 章：步态规划

步态规划的核心任务，是把“想往哪走”变成“每一只脚在什么时候落在哪里、以什么轨迹抬起来又放下去”。

6.1 步态规划通常包含四部分

- 相位调度：决定谁在 stance，谁在 swing
- 着地点规划：决定脚下一步踩哪
- 摆腿轨迹生成：决定 swing 相里足端怎么走
- 支撑相轨迹生成：决定 stance 相里足端怎么相对机身滑移

如果你要把它翻成软件模块，通常可以拆成：

- gait scheduler
- foothold planner
- swing trajectory generator
- stance controller / stance reference generator

这样每块职责就会清晰很多：

- scheduler 只管时序
- foothold planner 只管终点
- trajectory generator 只管从起点到终点怎么走
- stance controller 只管落地后的支撑行为

6.2 摆腿轨迹设计

摆腿轨迹至少要解决三个问题：

- 起点从哪里起
- 中间抬多高
- 终点落在哪里

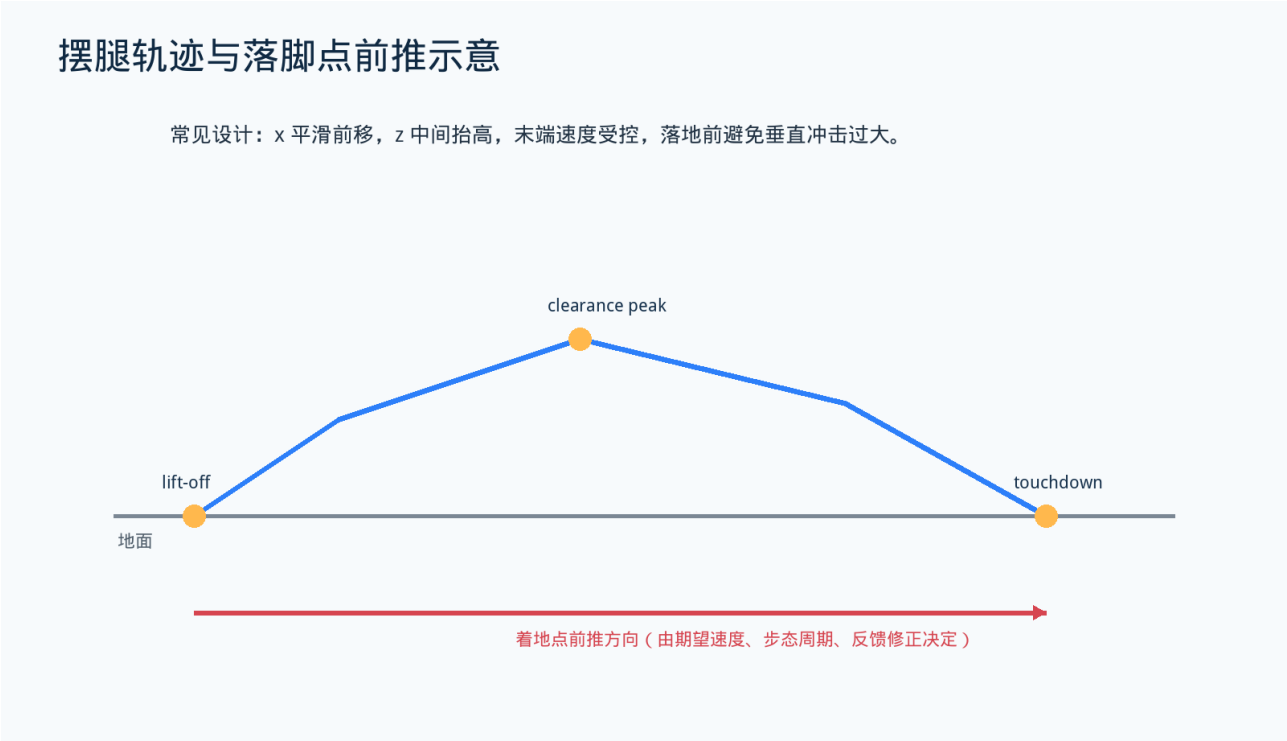
常见轨迹选择：

- 三次或五次多项式
- Bezier 曲线
- 正弦抬腿

最朴素但非常好用的一种想法是：

x(s): 从 lift-off 平滑走到 touchdown
y(s): 根据横向速度或转向需要变化
z(s): 中间抬高，起点和终点回到地面高度

其中 `s` 是 swing 相归一化相位，范围是 0 到 1。



摆腿轨迹与落脚点前推示意

真正工程上常要求它同时满足几件事：

- 起点连续：lift-off 时不要突然跳
- 终点连续：touchdown 前速度不要太大
- 中间抬脚足够高：避免蹭地
- 横向变化平滑：转弯时不要突然侧甩

如果你想做一个足够稳的最小版，建议边界条件至少设成：

$$s = 0:\ \ p = p_liftoff,\ \ v \approx current_foot_velocity$$
$$s = 1:\ \ p = p_touchdown,\ v \approx desired_touchdown_velocity$$

这样落地会明显比“只管位置、不管速度”平顺。

6.3 一个实用的摆腿高度经验

- 平地调试：先用较小抬腿高度
- 地毯、草地、轻障碍：适当增大
- 高速跑动：高度不能只靠拍脑袋，必须兼顾摆腿时间和落地冲击

太低：

- 容易蹭地

太高：

- 摆腿时间不够
- 落地冲击大

- 能耗高

所以摆腿高度不要单独调，而要和下面三个量一起看：

- 当前速度
- 当前 swing 时间
- 地形误差裕量

一个常见误区是“总觉得拖腿，就无脑加高”。这样很容易把问题从“蹭地”变成“摆腿赶不上 + 落地更重”。

6.4 着地点规划不只是“往前放”

一个成熟的着地点规划至少应考虑：

- 当前命令速度
- 当前实际速度
- 机身姿态和角速度
- 地形高度或可落脚区域
- 当前是哪种步态

当你只按命令速度前推落脚点，而完全不看真实状态时，机器人在受扰动后很容易越走越歪。

更直观一点说，落脚点本质上承担了三种职责：

- 提供下一拍的支撑几何
- 提供推进和减速的力学杠杆
- 帮助姿态和速度误差回收

所以一个“还算靠谱”的着地点规划，通常都至少带一点反馈味道，而不是纯前馈。

6.5 步长、步频、速度三者的关系

非常粗略的直觉可以理解为：

$$\text{速度} \approx \text{步长} \times \text{步频}$$

但工程上不能只盯这个公式，因为实际还受：

- duty factor
- 触地滑移
- 摆腿时间
- 摩擦和落地冲击
- 姿态振动

影响。

所以控制器里常见做法是：

- 给定速度区间
- 自动调步长、步频、抬腿高度、支撑时间
- 在不同速度区间使用不同步态

如果把这个关系再说工程化一点，就是：

- 想更快，不是只把步长拉大
- 也不是只把步频拉高
- 而是要一起考虑“腿来不来得及摆过去”和“落地能不能接得住”

6.6 地形感知步态规划

如果环境不是平地，步态规划要进一步加入：

- 地图或高程图
- 可达区域约束
- 足端避障
- 支撑稳定性评估

这时规划层通常不只输出一个 `vx`，而是输出：

- 期望机身轨迹
- 未来若干步着地点候选
- 每步的时序

即使你现在不做复杂地形，也建议先给架构留接口：

- 足端高度查询接口
- 可落脚区域过滤接口
- 障碍物或禁区过滤接口

这样以后从平地升级到台阶、斜坡、石块时，不需要推倒重来。

6.7 步态规划的工程分层

简单系统：

- 手工设定相位
- 规则法着地点
- 多项式摆腿

中等系统：

- 速度反馈修正着地点
- 摆腿轨迹自适应
- 有接触检测和时序修正

高级系统：

- 预瞄若干步
- 结合动力学约束
- 与 MPC / WBC 联动
- 同时考虑稳定性、效率、足端可达性、地形约束

判断你当前该停在哪一层，一个很好用的问题是：

- 你现在的主要瓶颈是“ 不会走 ”
- 还是“ 走得不稳 ”
- 还是“ 走得不快 ”
- 还是“ 遇到环境变化就崩 ”

不同瓶颈对应完全不同的优化优先级。

6.8 支撑相与摆动相控制器如何配合

初学者最容易把支撑相和摆动相都当成“ 给脚一个轨迹 ”。

其实两者目标完全不同：

- swing：要把脚安全、平滑地送到下一落脚点
- stance：要利用接触脚支撑身体、传递速度与姿态控制意图

因此一个最小实现里，通常会分成两条分支：

```
if leg_state == swing:
    foot_target = swing_trajectory(...)
else:
    foot_target = stance_rule(...)
```

更进一步一点：

- swing 更像“ 轨迹问题 ”
- stance 更像“ 支撑与反馈问题 ”

如果这两个概念不分开，你就很容易出现：

- 摆腿阶段还在做支撑修正
- 支撑阶段脚却在乱飘

而一旦分开，你会发现调试难度会下降很多，因为你能明确知道问题是：

- 摆腿轨迹错了
- 还是支撑规则错了

一个很重要的工程点是：两条分支虽然职责不同，但在 touchdown / liftoff 附近必须平滑衔接。

最常见的过渡处理包括：

- 对 stance / swing 权重做 blend
- 在 touchdown 前把足端速度收下来
- 在 touchdown 后逐步建立支撑锚点
- 在 liftoff 前逐步释放支撑约束

这一步做好了，控制器观感会从“ 腿在切换时抽搐 ”变成“ 腿自然接住和放开身体 ”。

6.9 对照开源项目，步态规划里最值得借鉴的是什么

如果只看前六章，最值得借鉴的其实不是某一个具体公式，而是这些项目共同采用的规划习惯。

从 Stanford Pupper 可以学到：

- 摆腿和支撑要明确分成两条逻辑
- stance 脚的目标常与机身期望速度方向相反
- swing 脚的落脚点要和支撑相扫过的距离闭环一致

从 CHAMP 可以学到：

- 摆腿高度、支撑时长、机身行走高度、CoM 偏移这些量必须显式可调
- 这些参数最好进配置，而不是散落在代码常数里

从 Unitree Guide 可以学到：

- 初学系统先把 FixedStand、Trotting 和基础键控速度走通
- 再往上叠加更复杂的路径、导航和高级控制

所以如果你现在要继续把前六章落到代码，最稳妥的路线通常不是“ 立刻引入更复杂算法 ”，而是：

```
先把模式切清楚
 -> 把 gait scheduler 做可视化
 -> 把 vx / vy / wz 送进足端层
 -> 把 swing / stance 分支彻底拆开
 -> 把关键步态参数做成显式配置
```

这条路线，本质上就是很多开源四足项目已经证明过的一条低风险工程路径。

6.10 本章源码对照阅读

本章最适合对照“ 摆腿 ”和“ 支撑 ”到底是不是两回事。

推荐对照：

- `Stanford Pupper Controller Description`
- 看 `stance controller` 和 `swing controller` 怎样明确分成两条逻辑
- `open-quadruped`

- 看 Bezier gait 与 body pose 怎样联动
- `CHAMP`
- 看 gait 参数配置怎样影响摆腿高度、支撑时间和机身高度
- 本地 `refcode/py_apple_quadruped_robot/Py-Apple Dynamics V4.0 二次开发教程.pdf`
- 用中文材料再对照一次“ 摆腿轨迹 + 落脚点 + 支撑逻辑 ”

这章读源码时，最值得盯的不是公式本身，而是下面三个接口：

- swing 轨迹起点从哪里来
- touchdown 终点由谁决定
- stance 状态下脚是不是被当成“ 锚点 ”或“ 支撑约束 ”

只要这三个接口清楚，你就能把“ 轨迹问题 ”和“ 支撑问题 ”从代码里拆开看。

第 7 章：步态切换

步态切换是把系统从“ 会走 ”拉到“ 像样 ”的关键一步。

很多系统明明单个步态能跑，但一切换就炸，原因通常不是单个步态错，而是切换没有设计成一个受控过程。

7.1 步态切换为什么难

因为切换改变的不是一个参数，而是一整套接触时序和运动风格：

- 相位差变了
- duty factor 变了
- 步频变了
- 摆腿高度变了
- 支撑时间变了
- 有时连姿态控制权重和期望机身高度都会变

所以切换不能只写成：

```
if speed > threshold:
    gait = trot
else:
    gait = walk
```

这只能叫“ 模式跳变 ”，不能叫“ 切换策略 ”。

7.2 触发切换的常见条件

- 速度区间
- 地形类型
- 负载变化

- 打滑或大扰动
- 能耗目标
- 操作员显式命令

工程上常见的切换触发逻辑是：

- 主条件：速度达到阈值
- 辅条件：姿态稳定、接触正常、关节未接近限位
- 安全条件：当前不是恢复状态或故障状态

7.3 切换时最重要的三个原则

- 不要在最脆弱的相位切
- 不要让参考量跳变过大
- 不要让执行层同时承受“ 相位变 + 足端突变 + 姿态突变 ”

7.4 实用切换策略一：参数渐变

不是一下换步态表，而是在一段时间内平滑过渡：

- 步频从 f1 渐变到 f2
- duty factor 从 d1 渐变到 d2
- 摆腿高度从 h1 渐变到 h2
- 相位差从 phi1 渐变到 phi2

优点：

- 简单
- 好调
- 对规则步态系统很有效

7.5 实用切换策略二：有限状态机

用专门的过渡态，而不是直接跳：

```
Stand -> WalkReady -> Walk -> TrotReady -> Trot
```

过渡态通常干什么：

- 把机身高度调到目标值
- 把速度指令斜坡到安全区间
- 对相位做重对齐
- 检查接触是否稳定

7.6 实用切换策略三：相位同步

很多时候切换失败，是因为新步态相位与当前腿部真实状态不一致。

比如：

- 旧步态里某腿还在 swing
- 新步态却把它当成 stance

解决思路：

- 读取当前每条腿的真实接触状态
- 把新步态的初始相位对齐到当前最接近的合法状态
- 必要时先进入一个短暂同步态

7.7 切换中的迟滞和防抖

如果速度阈值写成：

```
speed > 0.4 -> trot
speed <= 0.4 -> walk
```

那机器人会在阈值附近反复横跳。

正确做法是加迟滞：

- 从 walk 切 trot：`speed > 0.45`
- 从 trot 切 walk：`speed < 0.35`

7.8 步态切换的安全守则

- 切换前先限速
- 切换窗口内限制姿态命令
- 切换时增大足端安全高度
- 切换失败要能回退到保守步态
- 连续失败要能回到站立或恢复模式

7.9 一条实用的切换路线

- `Stand -> Static Walk`
- `Static Walk -> Crawl`
- `Crawl -> Trot`
- `Trot -> Fast Trot`
- `Fast Trot -> Bound`

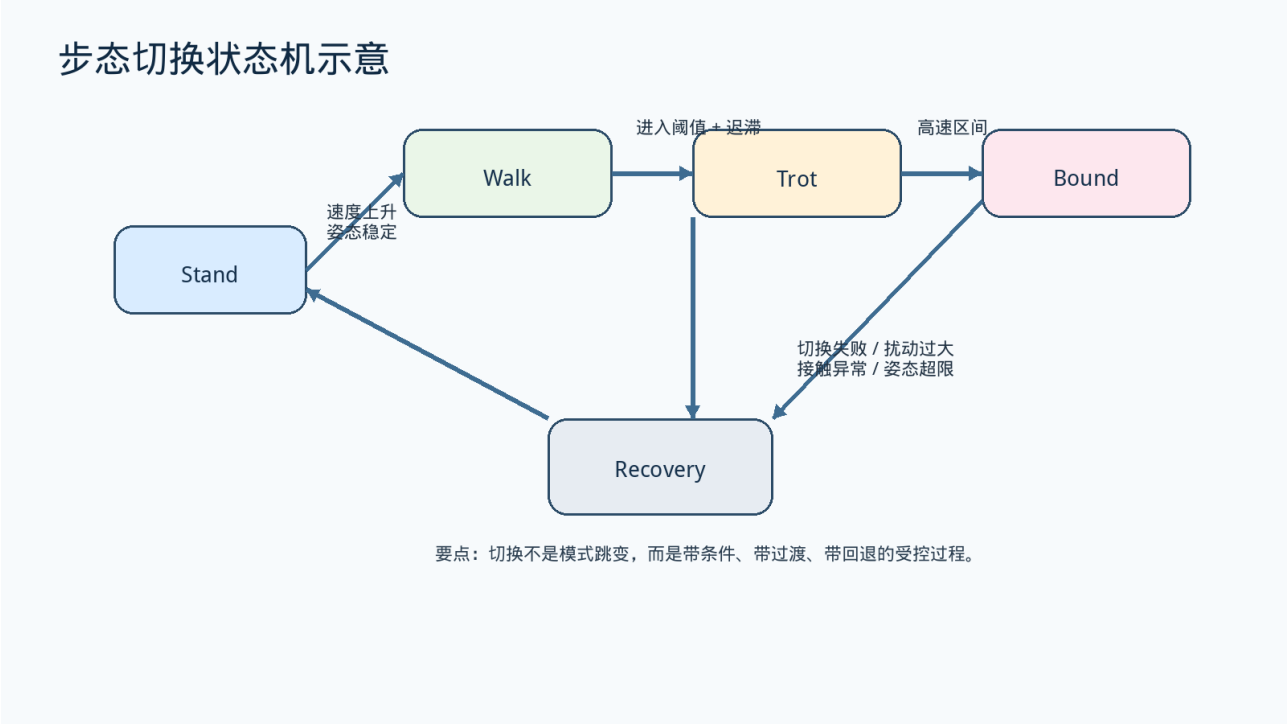
不要一开始就做：

- `Stand -> Bound`
- `Walk -> Gallop`

因为这会把问题叠加得太多。

7.10 步态切换评估指标

- 切换成功率
- 切换时间
- 切换期间姿态峰值误差
- 足端打滑次数
- 最大关节力矩
- 是否触发保护



步态切换状态机示意

7.11 一个 walk -> trot 切换落地流程

如果你要做第一条真正可用的步态切换，最推荐从 `walk -> trot` 开始。

一个很实用的落地流程是：

1. 检测速度是否达到 trot 进入阈值
2. 检查姿态是否稳定、接触是否正常
3. 进入过渡态，不立即切换
4. 在过渡态中逐步调步频、duty factor、摆腿高度
5. 把相位重对齐到最接近 trot 的合法状态
6. 切到 trot
7. 观察若干周期，如异常则回退

为什么这个流程适合新手：

- 它不是“突然跳”
- 它把切换分成了可观察的步骤

- 你可以逐步看是阈值错、相位错、还是参数渐变太快

7.12 本章源码对照阅读

步态切换最适合对照那些明确做了模式边界的项目。

推荐对照：

- `Unitree Guide`
- 优先看 `Passive / FixedStand / Trotting` 这种明确模式
- `CHAMP`
- 关注 bringup / config / gait 参数是怎样和模式边界隔开的
- `Stanford Pupper` / `StanfordQuadruped`
- 先只观察 `Stand -> Trot` 这类跨度不大的切换思路

读这章源码时，建议你只记录四个变量：

- 进入阈值
- 退出阈值
- 过渡态
- 回退条件

如果一个项目没有把这四件事写清楚，它大概率更偏 demo，而不是成熟切换系统。

第 8 章：从位置控制到阻抗控制、力控制、WBC

四足机器人控制并不是“只有一个正确答案”。不同阶段、不同硬件、不同目标，需要不同层级的控制方法。

8.1 位置控制

最容易上手：

- 给关节目标角
- 电机或舵机自己去跟踪

优点：

- 实现快
- 调试直观
- 非常适合前期把步态和 IK 跑通

缺点：

- 接触不柔顺
- 落地冲击较大

- 对地形误差敏感

8.2 阻抗控制

阻抗控制的核心思想是让脚或关节“ 像一个弹簧阻尼系统 ”。

它在四足机器人里非常常见，因为：

- 脚接触地面时更柔顺
- 轻微地形误差不会立刻变成巨大力矩
- 落地更自然

你可以把它理解成：

期望力 = 刚度 * 位置误差 + 阻尼 * 速度误差

8.3 力控制

当你想明确控制地面对每只脚的反作用力时，就进入力控制世界。

这在以下场景很重要：

- 快速运动
- 大扰动恢复
- 不同腿之间力分配
- 复杂地形接触

8.4 Whole-Body Control

WBC 的核心不是“ 一个很高级的黑盒 ”，而是：

- 同时考虑机身任务、足端任务、接触约束、关节约束
- 在一个统一优化框架里求出关节加速度、力矩或接触力

它擅长处理：

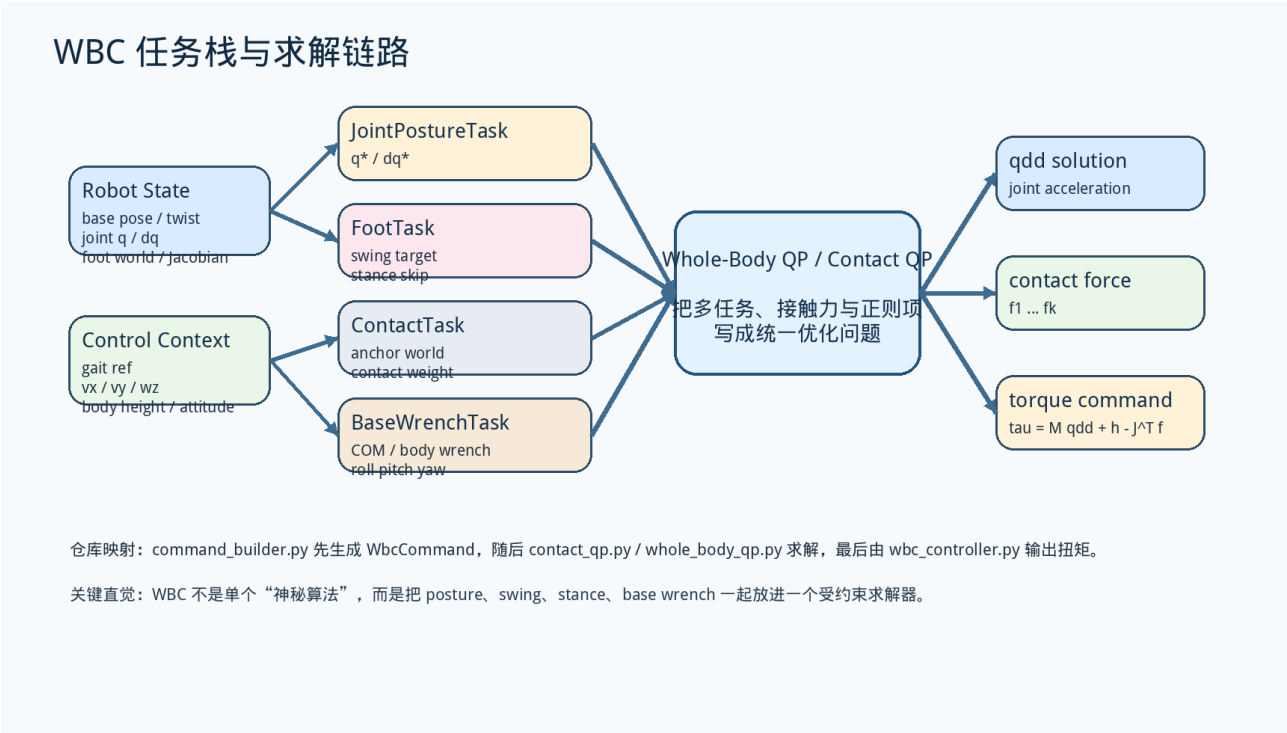
- 姿态任务
- 接触力分配
- 多任务优先级
- 约束统一

如果把它翻成工程语言，WBC 做的事情其实很朴素：

- 想让机身稳住
- 想让摆腿跟上目标
- 想让支撑腿别打滑
- 想让地面反力满足摩擦约束

- 想让输出力矩不要突然发疯

这些要求往往彼此冲突，所以 WBC
最常见的落地方式不是“ 手搓几条公式直接相加 ”，而是写成一个受约束优化问题。



WBC 任务栈与求解链路

8.4.1 为什么 WBC 往往最后写成 QP

因为 WBC 面对的不是一个目标，而是一组经常互相打架的目标：

- 机身姿态要稳
- 摆腿轨迹要跟踪
- 支撑足要贴住地面或围绕锚点稳定受力
- 接触力不能违反摩擦约束
- 关节加速度和力矩不能太激进

如果你把这些目标全都用“ 直接相加的 PD 力矩 ”处理，最常见的问题是：

- 摆腿和支撑腿任务在接触切换瞬间互相抢权
- 地面反作用力数学上可行、物理上不可信
- 力矩抖动大，落地冲击也大

QP（Quadratic Programming，二次规划）适合这里，是因为它天然处理这类问题：

最小化：若干个“ 误差平方项 ”的加权和
满足：若干个线性约束、盒约束、投影约束

翻成四足机器人的语言，就是：

- 把“ 机身 wrench 要尽量接近期望值 ”写成二次代价

- 把“摆腿加速度要尽量接近期望值”写成二次代价
- 把“支撑腿不要滑、法向力别为负、切向力别超过摩擦极限”写成约束或投影
- 把“别输出过猛的 qdd / f / tau”写成正则项

所以，WBC 不是因为“高级”才变成 QP，而是因为：

- 任务很多
- 约束很多
- 冲突很多
- QP 正好能把这些东西统一放进一个求解器

8.4.2 WBC 的输入、输出、决策变量

一个最常见的 WBC 回路，输入通常包括：

- 机器人当前状态：机身姿态、角速度、关节位置、关节速度
- 当前接触信息：哪条腿在支撑、哪条腿在摆动
- 任务参考：机身高度、roll / pitch / yaw、期望速度、足端目标
- 机器人模型：质量矩阵、偏置项、足端 Jacobian、质心位置

输出通常包括三类：

- 接触力 `f`
- 关节加速度 `qdd`
- 最终关节力矩 `tau`

在本仓库里，这些输入会先被整理成一个 `WbcCommand`，里面有几类任务对象：

- `JointPostureTask`
- `FootTask`
- `ContactTask`
- `BaseWrenchTask`
- `BaseOrientationTask`

你可以把它理解成：先把“我想让机器人做什么”描述清楚，再交给求解器。

从决策变量看，至少有两种常见写法：

第一种，只求接触力：

f = [f1x, f1y, f1z, f2x, f2y, f2z, ..., fkx, fky, fkz]^T

第二种，同时求关节加速度和接触力：

x = [qdd^T, f^T]^T

第一种更容易入门，因为你先只学“怎么把期望机身 wrench 分到几条支撑腿上”；第二种才是更完整的 whole-body QP。

8.4.3 先看最小接触力 QP

先只看支撑腿，不看摆腿和关节加速度。我们要做的事很简单：

- 目标机身 wrench 已知
- 当前有哪些腿在接触地面已知
- 每条腿施加多大接触力未知

定义：

```
f = [f1^T, f2^T, ..., fk^T]^T
w_des = [Fx, Fy, Fz, Mx, My, Mz]^T
```

如果第 `i` 条接触腿相对质心的位置是 `r_i`，那它对应的 wrench 贡献是：

```
[ fi ]
[ r_i x fi ]
```

把所有腿拼起来，就得到：

```
w = A f
A = [ I  I  ...  I
    [r1]x [r2]x ... [rk]x ]
```

这就是很多代码里叫 `wrench_map` 的东西：它负责把“每条腿的接触力”映射成“机身合力 + 合力矩”。

于是一个最小 contact QP 可以写成：

```
min_f  1/2 || Ww ( A f - w_des ) ||^2
    + 1/2 lambda_anchor || f - f_anchor ||^2
    + 1/2 lambda_share  || Sz f - fz_share ||^2
    + 1/2 lambda_reg    || f ||^2

s.t.  0 <= fz_i <= fz_max
      |fx_i| <= mu * fz_i
      |fy_i| <= mu * fz_i
```

这里每一项的物理意义分别是：

- 第一项：让总 wrench 尽量接近期望值
- 第二项：让支撑腿受力不要偏离“锚点力”太远
- 第三项：让几条支撑腿共享法向载荷，而不是一条腿吃满全部竖直力
- 第四项：避免求解出过大、过噪的接触力

在本仓库里，`mujoco_sim/wbc/contact_qp.py` 基本就是这个思路：

- `wrench_map`：把接触力拼成机身 wrench
- `anchor_targets`：锚点力目标
- `normal_share_targets`：法向载荷分配目标

- `project_contact_force()`：把每条腿的力投影回可行摩擦域

8.4.4 再看最小 whole-body QP

只求接触力还不够，因为机器人还有：

- 关节姿态任务
- 摆腿轨迹任务
- 支撑腿稳定任务
- 机身姿态 / 高度 / COM 任务

所以更完整的做法是把 `qdd` 和 `f` 放进同一个变量里：

```
x = [qdd^T, f^T]^T
```

然后把各类任务都写成“目标加速度”或“目标 wrench”的误差平方项。

一个非常常见的 whole-body QP 形式是：

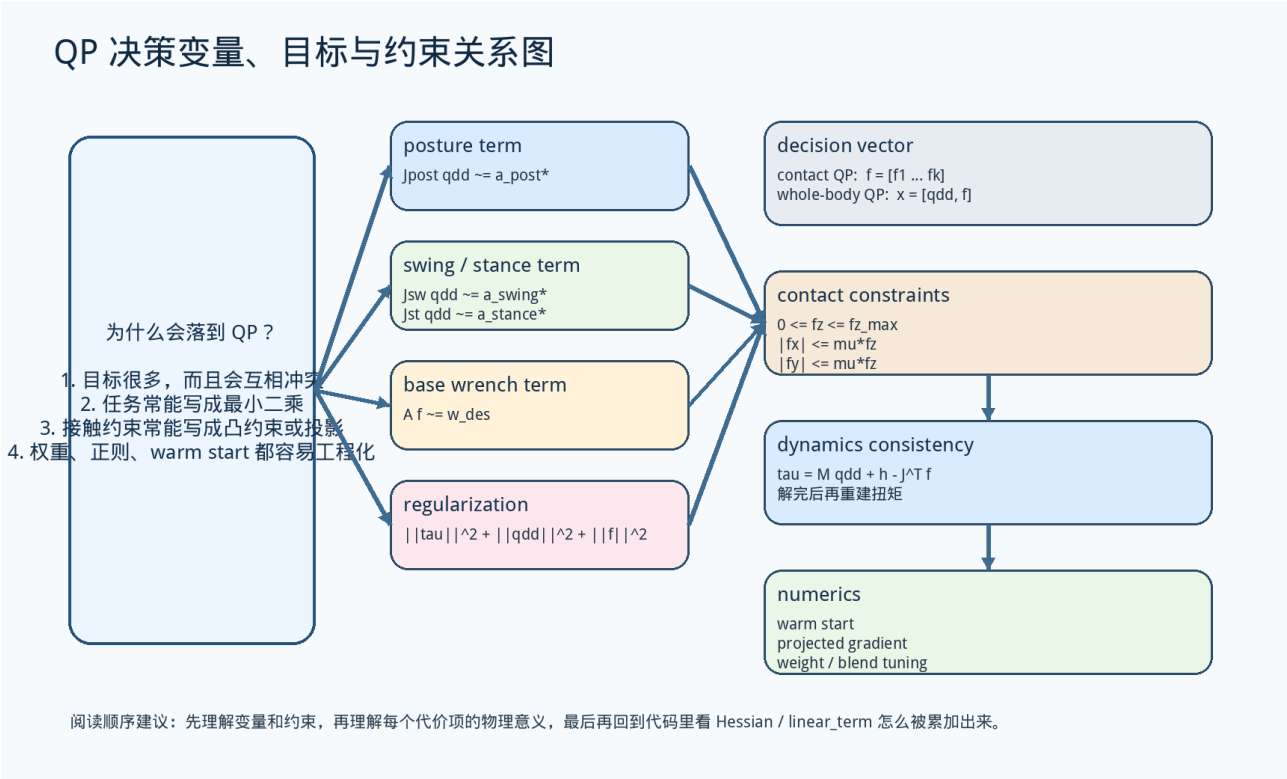
```
min_x  1/2 || Wpost   ( Jpost qdd - a_post*   ) ||^2
     + 1/2 || Wswing  ( Jsw   qdd - a_swing*  ) ||^2
     + 1/2 || Wstance ( Jst   qdd - a_stance* ) ||^2
     + 1/2 || Wwrench ( A f       - w_des   ) ||^2
     + 1/2 lambda_tau || M qdd + h - Jc^T f ||^2
     + 1/2 lambda_qdd || qdd ||^2
     + 1/2 lambda_f   || f ||^2
```

其中：

- `a_post*`：关节姿态任务希望得到的关节加速度
- `a_swing*`：摆腿末端希望得到的足端加速度
- `a_stance*`：支撑腿围绕锚点的稳定加速度
- `w_des`：机身希望得到的合力和合力矩
- `M qdd + h - Jc^T f`：动力学一致性对应的力矩项

把这些项放在一起后，求解器会自动做折中：

- 姿态很差时，优先稳住机身
- 摆腿快落地时，给摆腿任务更多权重
- 支撑少时，接触力约束更重要
- 力矩太大时，正则项会把动作收回来



QP 决策变量、目标与约束关系图

本仓库的 `mujoco_sim/wbc/whole_body_qp.py` 就是沿这个框架写的。它直接把变量堆成：

```
x = [qdd, f_contact]
```

然后依次累加：

- posture 项
- swing 项
- stance 项
- base wrench 项
- torque / qdd / force 正则项

最后再从解里恢复最终力矩。

8.4.5 约束、正则项、warm start 各自干什么

初学者最容易把这些东西都看成“数学装饰”，其实它们全是工程稳定性的核心。

先看约束：

- `fz >= 0`：脚只能推地，不能“拉地”
- `|fx| <= mu*fz`、`|fy| <= mu*fz`：切向力不能超过摩擦能力
- 接触切换时，只对真正处于支撑相的腿求接触力

再看正则项：

- `||f||^2`：避免接触力过大、过抖

- `||qdd||^2`：避免关节加速度过猛
- `||tau||^2`：避免最终输出力矩过激

再看 warm start 和投影：

- warm start：用上一拍的 `qdd` / `f` 当本拍初值，减少跳变
- 投影：每次迭代后都把力投影回摩擦域，防止数值结果漂出物理可行域
- blend：在接触数量变化、摆腿快落地时，让控制器别突然完全切到另一套解

因此，一个“能跑起来的 WBC”不只是公式正确，还要数值上稳定、接触切换时平滑、调参时可解释。

8.4.6 本仓库里的 WBC/QP 主链路

如果你把本仓库对应到上面的公式，主链路非常清晰：

1. `mujoco_sim/wbc/command_builder.py`

- 读取机器人状态、步态参考、目标速度和姿态参考
- 生成 `JointPostureTask`、`FootTask`、`ContactTask`、`BaseWrenchTask`
- `yaw_aware_swing_target()` 里还带了一个很小的 foothold QP，用来调摆腿落脚点

2. `mujoco_sim/wbc/contact_qp.py`

- 用 `wrench_map` 把接触力映射成机身 wrench
- 用 `anchor_targets`、`normal_share_targets` 做锚点和载荷分配正则
- 用 `project_contact_force()` 保证每条腿接触力满足摩擦约束

3. `mujoco_sim/wbc/whole_body_qp.py`

- 把决策变量写成 `[qdd, f_contact]`
- 通过 `accumulate_quadratic_term()` 累加各任务的 Hessian 和 linear term
- 调用 `solve_whole_body_qp()` 迭代求解
- 再用 `tau = M qdd + h - J_c^T f` 恢复关节力矩

4. `mujoco_sim/wbc_controller.py`

- 在 `compute_torques()` 里先算一套基线任务扭矩
- 再算一套 QP 解
- 通过 `adaptive_whole_body_qp_blend()` 做自适应混合

这说明当前仓库里的控制器，更准确地说是：

- 一条显式任务扭矩链
- 一条 contact / whole-body QP 链
- 再加一个按接触状态自适应调节的混合层

理解这一点非常重要，因为它能帮你区分三件事：

- 哪些行为来自任务 PD

- 哪些行为来自接触力优化
- 哪些行为来自两条路线之间的 blend

8.5 MPC 与 WBC 的典型关系

在很多成熟四足系统里：

- MPC 负责较长时间尺度的反作用力或质心运动规划
- WBC 负责较短时间尺度的全身一致性和力矩分配

一个经典思路是：

MPC: 规划未来若干步的 COM / GRF
WBC: 在当前时刻把这些目标落实成关节层命令

8.6 什么时候该升级控制方法

- 如果你连站立都不稳，不该先上 WBC
- 如果你的位置控制已经能在平地稳定 trot，但地形稍变就崩，可以考虑阻抗
- 如果你要做高速、跳跃、复杂地形、强扰动恢复，应考虑力控制、WBC、MPC

8.7 新手最需要避免的误区

- 把高级算法当银弹
- 没有可靠状态估计就直接做复杂优化
- 没有日志与调试工具就堆更多数学

工程顺序永远应该是：

- 先可观测
- 再可重复
- 再可调
- 最后再上更复杂算法

8.8 本章源码对照阅读

如果你准备从“ 前六章规则控制 ”跨到“ MPC / WBC / QP ”，最推荐的源码对照顺序是：

- `MIT Cheetah Software`
- 先看整体软件栈，不要一开始就钻单个 solver
- `TopHillRobotics/quadruped-robot`
- 看仿真、实机和控制器如何被组织在同一套工程里
- `Quadruped-PyMPC`
- 专门看 MPC 的状态、决策变量和约束
- `WBC-quadruped-DOB`
- 专门看 whole-body control、外扰观测和 Gazebo 场景

- 最后再回到当前仓库
- `mujoco_sim/wbc_controller.py`
- `mujoco_sim/wbc/`

正确读法不是“从公式跳到代码”，而是：

```
先看任务分层
 -> 再看变量和约束
 -> 再看 solver
 -> 最后看如何恢复成扭矩或执行命令
```

这样你才不会被高级控制部分的数学符号直接淹没。

第 9 章：状态估计与传感器

没有状态估计，就没有稳定控制。

9.1 你至少需要哪些反馈

- IMU：roll / pitch / yaw，角速度，加速度
- 关节编码器：关节位置、速度
- 足接触信息：脚是否着地、接触是否可靠
- 机身速度或里程计：用于速度闭环和着地点修正

9.2 为什么 IMU 很关键

姿态控制、步态切换、落地判断、扰动恢复，几乎都离不开 IMU。

常见用法：

- roll/pitch 直接进入姿态闭环
- yaw 用于航向控制或世界系对齐
- 角速度用于阻尼项和状态预测

9.3 足接触检测有多重要

如果控制器以为脚已经落地，但实际上还没落地：

- stance 控制会失真
- 力分配会错
- 姿态估计可能漂

如果控制器以为脚没落地，但实际已经撞地：

- 摆腿轨迹会被强行继续
- 容易打脚或绊倒

9.4 常见状态估计结构

- 简单版：IMU + 关节编码器 + 接触规则
- 中等版：机身状态 EKF
- 高级版：IMU、腿里程计、视觉或激光融合

9.5 状态估计中的工程重点

- 传感器时间同步
- 延迟补偿
- 接触状态可信度
- 低通滤波与响应速度平衡
- 世界系与机体系转换一致

9.6 初学者可实现的最小状态估计链

对零基础来说，完全没必要一上来就做很重的融合系统。

你可以先做一条最小状态估计链：

- IMU 给姿态
- 关节编码器给关节角
- 接触规则给哪条腿在地上
- 足端与机身关系给一个粗略速度或位姿校正

这个阶段最关键的不是“估得最精确”，而是：

- 数据方向对
- 时间戳对
- 滤波不过头
- 能稳定喂给控制器

等这条链路稳了，再逐步升级到：

- EKF
- 腿里程计
- 视觉或激光融合

9.7 本章源码对照阅读

状态估计章节最推荐你带着“观测量从哪里来、最终喂给谁”这两个问题去读项目。

推荐对照：

- `Unitree Guide`、`CHAMP`、`DingoQuadruped`
- 优先找 IMU、joint state、contact / odometry 这三类输入是怎样进入控制器的

- `WBC-quadruped-DOB`
- 看 Gazebo 下观测量怎样服务 whole-body control
- 本地中文项目
- `refcode/tinymal_b_quadruped_robot/README.md`
- `refcode/py_apple_quadruped_robot/README.md`
- 用整机视角理解 IMU 安装、线缆、电源和控制入口对状态估计链的影响

读完后你最好能回答：

- 哪些量是原始传感器数据？
- 哪些量是控制器真正需要的估计状态？
- 接触状态是“直接测出来的”，还是“规则 + 滤波 + 估计”出来的？

第 10 章：完整控制架构

一套清晰的控制架构，通常比单个炫酷算法更重要。

10.1 推荐的分层架构

```
任务层
  路径、导航、目标点、操作命令

运动层
  期望速度、期望姿态、步态管理、切换策略

轨迹层
  足端着地点、摆腿轨迹、支撑轨迹

控制层
  IK / 位置控制 / 阻抗 / WBC / MPC

执行层
  电机、舵机、驱动器

估计层
  IMU、关节、接触、里程计融合
```

10.2 为什么要分层

因为不同问题的时间尺度完全不同：

- 路径规划：几十毫秒到几百毫秒
- 步态和足端轨迹：几毫秒到几十毫秒
- 关节控制：1 毫秒量级甚至更快

如果把所有逻辑都塞进一个巨大循环里，后果就是：

- 难调试

- 难替换
- 难定位问题
- 难扩展新步态或新传感器

10.3 典型运行周期

- 高层导航：10 Hz 到 30 Hz
- 步态管理与足端轨迹：50 Hz 到 200 Hz
- 低层关节控制：200 Hz 到 1000 Hz

10.4 实际工程中的模式管理

成熟系统一般会有明确模式：

- Passive
- Stand
- Walk
- Trot
- Recovery
- Emergency Stop

而不是只用一个变量 `mode = 1` 草草了事。

> 系统成熟度的一个分水岭，就是你是否已经从“一个大 while 循环”走到了“有模式、有边界、有回退、有回归”的工程结构。

10.5 每一层到底输入什么、输出什么

新手读控制架构时最容易困惑的一点是：

- 我知道有很多层，但每一层到底吃什么、吐什么？

你可以把它记成下面这张最小表：

任务层输出：期望速度、期望方向、期望姿态

运动层输出：步态模式、phase、摆腿/支撑状态

轨迹层输出：每条腿的足端目标或落脚点目标

控制层输出：关节角、力矩或电机命令

估计层输出：姿态、速度、接触、关节状态

- 任务层输入：目标点、路径、操作者命令
- 运动层输入：期望速度与姿态
- 轨迹层输入：步态模式与速度目标
- 控制层输入：足端目标与状态反馈
- 估计层输入：传感器原始数据

一旦你能把每层输入输出说清，整个系统就不会再显得模糊。

10.6 本章源码对照阅读

如果你已经不满足于“会跑”，而是想真正学会“怎么组织一套系统”，这一章最适合跨项目对照。

推荐顺序：

- `StanfordQuadruped`
- 看一个“小而完整”的整机控制项目是什么样
- `CHAMP`
- 看 ROS 化 package split、bringup、配置和仿真
- `TopHillRobotics/quadruped-robot`
- 看仿真 / 实机 / 控制器边界
- `MIT Cheetah Software`
- 看研究级软件栈的整体味道
- 再回到当前仓库
- `mujoco_sim/controllers/`
- `mujoco_sim/wbc/`
- `mujoco_sim/backends/`
- `mujoco_sim/runtime/`
- `mujoco_sim/editor/`
- `tests/`

最推荐的读法不是横着乱翻，而是按下面顺序：

```
输入
 -> 模式
 -> 轨迹
 -> 控制
 -> 后端
 -> 回归测试
```

只要你能把一个项目按这个顺序读通，迁移到另一个项目时就会快很多。

第 11 章：从仿真到实机

从仿真到实机的最大区别，不是“电机会更强”，而是现实世界会把所有被你忽略的细节全部放大。

11.1 仿真里容易被忽略的东西

- 地面摩擦不准
- 电机延迟和带宽限制

- 编码器噪声
- 结构弹性
- 安装误差
- 线缆和电源约束

11.2 正确的迁移顺序

- 先在仿真做站立
- 再做小范围姿态调节
- 再做单腿摆动
- 再做低速 walk
- 再做低速 trot
- 最后再做高速步态和切换

11.3 实机调试三原则

- 任何新功能先降速、限幅、加保护
- 永远保留一键回站立或急停
- 先验证可重复，再提高性能

11.4 从仿真迁移时最该检查的参数

- 关节零位
- 腿长参数
- 机身质量与质心
- 电机方向和符号
- IMU 安装方向
- 接触阈值
- 关节限位

11.5 实机上线前的最后 12 项安全检查

在第一次把新控制器放到实机前，建议逐项过下面 12 条：

- 急停是否真的有效
- 能否一键回到站立
- 关节方向和零位是否确认
- 默认站姿是否对称
- 高度是否留有安全余量
- 关节限位是否配置
- 速度命令是否有限幅和斜坡

- 切换逻辑是否暂时关闭或保守
- 日志是否打开
- IMU 方向是否核对
- 供电和线缆是否安全
- 周围是否有足够安全空间

如果这 12 条没过，不建议直接测试更激进的步态。

11.6 本章源码对照阅读

从仿真到实机这一章，最适合对照那些同时覆盖 bringup、Gazebo 或整机装配的项目。

推荐对照：

- `CHAMP`、`DingoQuadruped`
- 看 ROS / Gazebo bringup 的组织方式
- `TopHillRobotics/quadruped-robot`、`WBC-quadruped-DOB`
- 看仿真场景如何接到控制器
- 本地中文项目
- `refcode/tinymal_b_quadruped_robot/README.md`
- `refcode/py_apple_quadruped_robot/开源菠萝四足机器人项目--上手指南（先看我）.pdf`
- 重点看装配、控制器说明和上手机制

读这章时最有价值的，不是比较谁算法更高级，而是比较：

- 哪些参数在仿真里能乱来，实机里绝对不能乱来
- 哪些安全措施项目里是“默认开启”的
- 哪些项目把实机 bringup 拆成了明确步骤

第 12 章：结合本仓库做系统实践

如果你要结合当前仓库学习，可以把它理解为一条已经被分层过的教学链路。

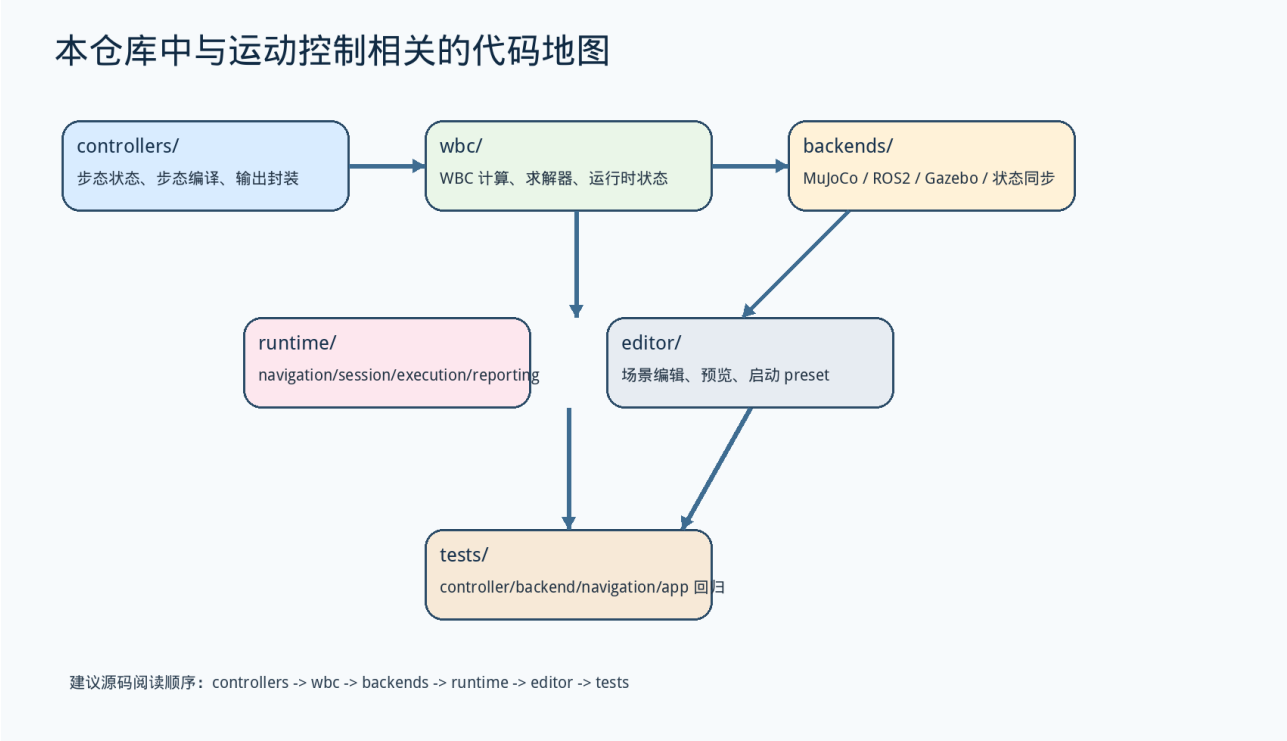
12.1 推荐阅读顺序

- `README.md`
- `mujoco_sim/README.md`
- `docs/DUAL_BACKEND_ARCHITECTURE.md`
- `docs/STRUCTURE_OPTIMIZATION_ROADMAP.md`

12.2 与教程概念对应的仓库模块

- 控制编排：`mujoco_sim/controllers/padog_controller.py`

- 控制拆分：`mujoco_sim/controllers/padog_step.py`
- 步态编译：`mujoco_sim/controllers/padog_compile.py`
- 输出封装：`mujoco_sim/controllers/padog_output.py`
- 高级低层控制：`mujoco_sim/wbc_controller.py` 与 `mujoco_sim/wbc/`
- 仿真与回归后端：`mujoco_sim/backends/mujoco_runtime.py`
- ROS2 / Gazebo 后端：`mujoco_sim/backends/ros2_gazebo.py`
- 传输层：`mujoco_sim/backends/ros2_transport.py`
- 状态同步：`mujoco_sim/backends/gazebo_state_sync.py`
- 场景编辑器：`mujoco_sim/nav_scene_editor.py`
- 回归测试：`tests/backends/`、`tests/controllers/`、`tests/navigation/`、`tests/apps/`



本仓库中与运动控制相关的代码地图

12.3 一个很适合本仓库的学习顺序

- 用 `mujoco` backend 做站立、姿态、步态调试
- 用 `ros2_gazebo/loopback` 走完整链路回归
- 再切 `ros2_gazebo/live` 对接真实 ROS2/Gazebo

12.4 建议的仓库内练习题

- 练习 1：只改姿态目标，观察足端和关节的变化
- 练习 2：固定步态，只改 `vx / vy / wz`
- 练习 3：给 trot 增加速度斜坡和限幅
- 练习 4：实现 walk 到 trot 的迟滞切换

- 练习 5：记录一次失败切换的日志并定位根因
- 练习 6：比较 `mujoco` 与 `ros2_gazebo` 观测差异

12.5 把本仓库和外部开源项目对照着读

如果你准备把当前仓库和外部项目一起读，最推荐下面这条路线：

- 第 1 轮：`Stanford Pupper Docs` + 本仓库 `mujoco_sim/controllers/`
- 目标：把最小控制链读顺
- 第 2 轮：`CHAMP` + 本仓库 `mujoco_sim/backends/`
- 目标：体会配置化和 ROS / Gazebo 接口边界
- 第 3 轮：`Unitree Guide` + 本仓库步态与模式逻辑
- 目标：理解 mode、切换和速度命令入口
- 第 4 轮：`TopHillRobotics` / `Quadruped-PyMPC` / `WBC-quadruped-DOB` + 本仓库 `mujoco_sim/wbc/`
- 目标：把高级控制从“ 名词 ”读成“ 变量和模块 ”

如果你照这个顺序读，你会明显感觉：

- 外部项目帮你建立“ 别人的成熟抽象 ”
- 当前仓库帮你建立“ 你手头这套系统怎么落地 ”

两者结合起来，学习速度会比只盯着一个仓库快很多。

第 13 章：调参与排障手册

13.1 如果机器人站不稳

先查：

- 零位是否对
- 默认站姿是否对称
- IMU 方向是否对
- 姿态补偿增益是否过大
- 足端目标是否越界

13.2 如果机器人慢走还行，一加速就崩

优先查：

- 步频是否太低
- 步长是否过大
- 摆腿高度是否不够
- stance / swing 时间分配是否合理

- 速度命令是否没有斜坡

13.3 如果 trot 能跑，但切换失败

优先查：

- 是否有迟滞
- 是否有过渡态
- 切换瞬间相位是否对齐
- 切换窗口内姿态和速度是否限幅
- 是否把落脚点、步频、抬腿高度同时跳变了

13.4 如果落地冲击很重

可能原因：

- 摆腿末端速度太大
- 抬腿轨迹太尖
- 接触检测太晚
- 阻抗太硬
- 机身下落速度太大

13.5 如果脚容易滑

可能原因：

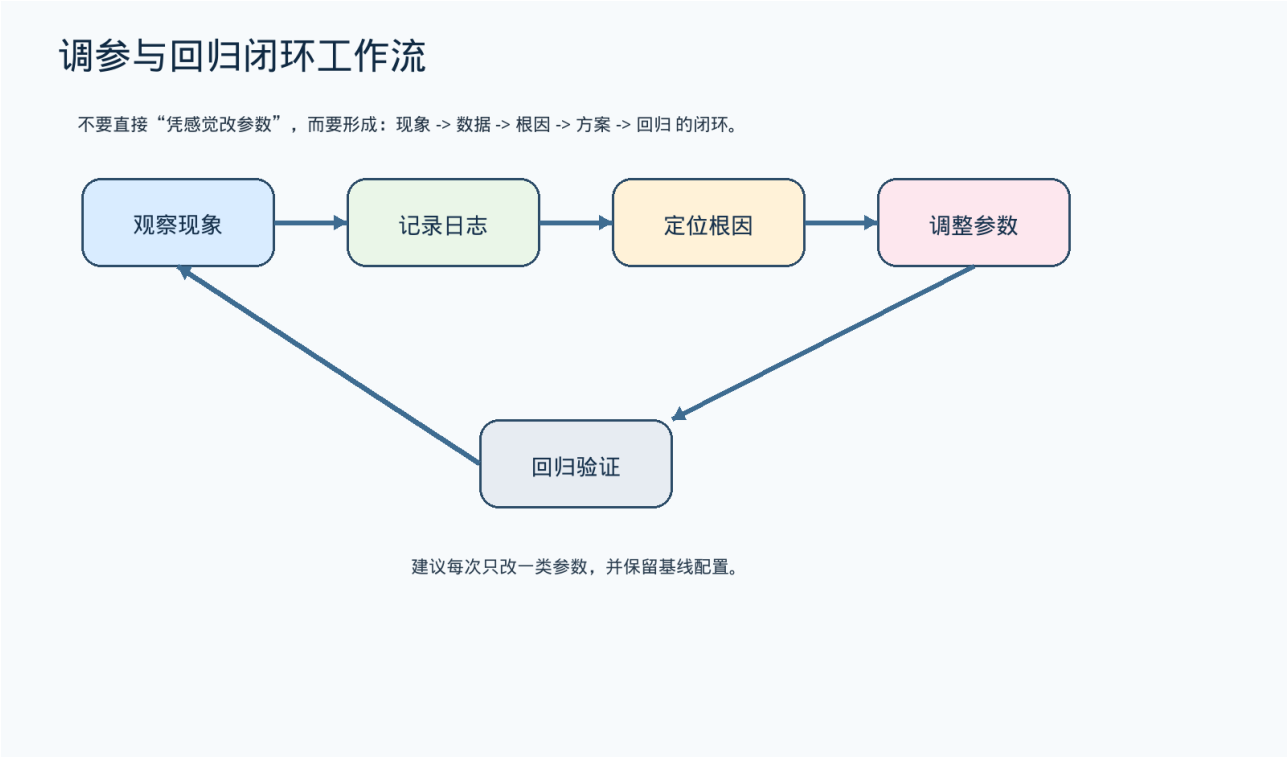
- 着地点太激进
- 支撑相足端速度设错
- 地面摩擦估计不准
- 身体姿态导致法向力不足
- 控制器对实际速度误差没有反馈修正

13.6 强烈推荐记录的日志

- 用户命令速度
- 实际机身速度
- roll / pitch / yaw
- 每条腿的 phase
- 每条腿接触状态
- 每条腿足端目标
- 关节目标和实际值
- 切换状态机状态

13.7 最有用的可视化

- phase 曲线
- 足端轨迹
- 机身姿态误差
- 接触时刻标记
- 速度命令与实际速度对比
- 切换前后参数渐变曲线



调参与回归闭环工作流

第 14 章：从入门到精通的训练计划

14.1 入门阶段：先把基本链路跑通

目标：

- 能看懂坐标系
- 能做默认站姿
- 能做姿态补偿
- 能做单腿摆动

你应该完成：

- 一版可靠 IK
- 一版稳定站立

- 一版最简单的脚尖轨迹

14.2 初级阶段：会走

目标：

- 做出静态步行
- 能响应 `vx / vy / wz`
- 能解释步长、步频、duty factor

你应该完成：

- 一个基础 gait scheduler
- 一个基础摆腿轨迹生成器
- 一个基础速度控制接口

14.3 中级阶段：会跑

目标：

- 跑通 trot
- 支持速度斜坡
- 支持低速到中速连续变化

你应该完成：

- trot phase 设计
- 落脚点前推
- 姿态与速度闭环初步配合

14.4 进阶阶段：会切换

目标：

- walk / trot 之间可切
- 切换成功率高
- 切换过程中姿态峰值可控

你应该完成：

- 有限状态机
- 参数渐变器
- 切换迟滞与回退逻辑

14.5 高级阶段：会做高级控制

目标：

- 理解阻抗、力控制、WBC、MPC 的边界
- 能根据平台能力选择控制层级

你应该完成：

- 一个更可靠的状态估计链路
- 一个更好的接触检测机制
- 一套可解释的全身控制接口

14.6 精通阶段：会做系统优化

精通不是“公式背得多”，而是你能同时回答下面这些问题：

- 为什么这个步态适合这个速度区间
- 为什么这次切换会失败
- 为什么这个平台更适合阻抗而不是纯位置控制
- 为什么姿态稳了但速度跟踪变差
- 为什么仿真能跑实机不能跑
- 如何把控制链路拆成便于维护和回归的模块

附录 A：关键公式与直觉

A.1 机身速度到支撑足速度

在机身坐标系下，一个常见直觉是：

$$v_foot_stance \approx -v_body_cmd - wz \times r_foot$$

意思是：

- 如果身体想往前走，支撑脚相对身体就要向后扫
- 如果身体想左转，脚还要额外承担旋转分量

A.2 简化着地点前推

$$p_next \approx p_nominal + 0.5 * T_stance * v_cmd$$

A.3 一个常见摆腿高度函数

$$z(s) = h * \sin(pi * s),\ s \text{ in } [0, 1]$$

A.4 速度变化的工程约束

$$v_cmd(t) = clamp(v_ref, v_prev - a_max*dt, v_prev + a_max*dt)$$

附录 B：步态切换伪代码

```
if mode == WALK:
    if speed > trot_enter and posture_ok and contacts_ok:
```

```
        transition = WALK_TO_TROT

    if transition == WALK_TO_TROT:
        blend.phase_offset = lerp(walk_phase, trot_phase, alpha)
        blend.duty      = lerp(walk_duty, trot_duty, alpha)
        blend.frequency  = lerp(walk_freq, trot_freq, alpha)
        blend.clearance  = lerp(walk_h, trot_h, alpha)

        if alpha >= 1.0 and phase_aligned:
            mode = TROT
            transition = NONE

    if mode == TROT:
        if speed < trot_exit:
            transition = TROT_TO_WALK
```

这段伪代码真正想表达的是：

- 切换要有条件
- 切换要有过程
- 切换要有完成判据
- 切换要有回退逻辑

附录 C：从零到精通的最小实践清单

- 机器人能稳定站立 30 秒
- 高度和姿态调节不抖
- 单腿摆动不会带倒整机
- static walk 连续成功
- trot 连续成功
- `vx / vy / wz` 可解释可调
- walk 到 trot 切换成功率高
- 有完整日志
- 有回归测试
- 有从仿真到实机的安全迁移步骤

附录 D：术语表

- Base：机身主体
- COM：质心
- GRF：地面对足端的反作用力
- Support Polygon：支撑多边形
- Stance：支撑相

- Swing：摆动相
- Duty Factor：支撑相占比
- Phase Offset：不同腿之间的相位差
- Stride Length：步长
- Cadence：步频
- Foothold：落脚点
- Gait Scheduler：步态调度器
- FK：正运动学
- IK：逆运动学
- Impedance Control：阻抗控制
- WBC：全身控制
- MPC：模型预测控制
- Estimator：状态估计器
- Hysteresis：迟滞
- Recovery Mode：恢复模式

附录 E：建议实验路线

E.1 零基础 7 步实验路线

- 实验 1：确认坐标系、腿编号、关节方向
- 实验 2：固定站立，调高度与默认姿态
- 实验 3：单腿摆动，验证足端轨迹与 IK
- 实验 4：四腿慢走，验证 phase 和 duty factor
- 实验 5：加入 `vx / vy / wz` 速度命令
- 实验 6：实现 walk 到 trot 的切换与迟滞
- 实验 7：做一次完整回归，记录切换成功率与姿态峰值

E.2 每一步的验收标准

- 看得懂：知道这一步要解决什么问题
- 跑得通：行为可重复
- 可解释：知道为什么会失败
- 可回归：改参数后能重跑验证

附录 F：实验记录模板

实验名称：
日期：
软件版本：

场景：
步态：
速度命令：

现象：
-

日志结论：
-

怀疑根因：
-

本轮改动：
-

回归结果：
- 成功率
- 姿态峰值
- 是否打滑
- 是否触发保护

下一轮计划：
-

附录 G：真正做到“精通”前你要能回答的 20 个问题

- 为什么你的默认站姿是稳定的？
- 为什么这套默认足端位置不会把关节推到奇异位形？
- 为什么当前速度区间更适合 walk 而不是 trot？
- 为什么这次切换要加迟滞？
- 为什么你的落脚点公式里需要速度反馈项？
- 为什么摆腿高度增大后反而更容易摔？
- 为什么位置控制在平地够用，但上台阶就不够了？
- 为什么 IMU 噪声会直接影响姿态稳定性？
- 为什么接触检测错误会导致支撑相控制失真？
- 为什么 trot 常常是第一个被做出来的动态步态？
- 为什么同一套参数在仿真能跑，实机却不行？
- 为什么要分层，而不是写一个超大的控制循环？
- 为什么 WBC 需要明确任务优先级？
- 为什么 MPC 与 WBC 常常配合使用？
- 为什么回归测试对运动控制也很重要？
- 为什么只看最终是否摔倒，不足以判断控制器优劣？
- 为什么切换成功率是核心指标之一？

- 为什么足端轨迹要平滑，而不能让落脚点突变？
- 为什么速度命令必须做斜坡和限幅？
- 为什么成熟系统一定需要恢复模式？

附录 H：源码阅读导航

如果你要把这份教程和当前仓库一起学，推荐按下面顺序阅读源码：

在进入这个附录前，建议先把各章末尾新增的“ 本章源码对照阅读 ”过一遍：

- 那些小节负责把外部开源项目和本教程概念对齐
- 这个附录负责把焦点重新收回当前仓库

H.1 第一遍：只看主链路

- `mujoco_sim/controllers/padog_controller.py`
- `mujoco_sim/controllers/padog_step.py`
- `mujoco_sim/controllers/padog_compile.py`
- `mujoco_sim/controllers/padog_output.py`

目标：

- 明白控制器如何接收姿态、速度、步态输入
- 明白控制器如何整理成最终输出命令

H.2 第二遍：看高级低层控制

- `mujoco_sim/wbc_controller.py`
- `mujoco_sim/wbc/`

目标：

- 明白 WBC 如何拆成 command builder、solver、runtime state
- 明白 WBC 在整条控制链中的位置

H.3 第三遍：看后端与状态同步

- `mujoco_sim/backends/ros2_gazebo.py`
- `mujoco_sim/backends/ros2_transport.py`
- `mujoco_sim/backends/gazebo_service_bridge.py`
- `mujoco_sim/backends/gazebo_state_sync.py`

目标：

- 明白控制命令如何发到 backend
- 明白观测是如何从 joint_state / imu / odom / model_state 汇总回来的

H.4 第四遍：看运行时与回归

- `mujoco_sim/runtime/`
- `tests/controllers/`
- `tests/backends/`
- `tests/navigation/`
- `tests/apps/`

目标：

- 明白整套系统如何被回归测试保护
- 明白结构清晰为什么会直接降低调试难度

附录 I：参数调优速查表

I.1 姿态类参数

- 机身高度：过低容易蹭地，过高容易失稳
- roll/pitch 增益：过低补不动，过高振荡
- yaw 控制：建议最后再开大

I.2 步态类参数

- duty factor：大则稳，小则快
- phase offset：决定步态接触节律
- 步频：过低响应慢，过高接触噪声大
- 步长：过大容易拖腿、滑脚、超可达域

I.3 摆腿类参数

- 抬腿高度：过低蹭地，过高耗能且冲击大
- 落脚点前推：过小速度跟不上，过大落地不稳
- 摆腿末端速度：过快时落地冲击会显著增加

I.4 切换类参数

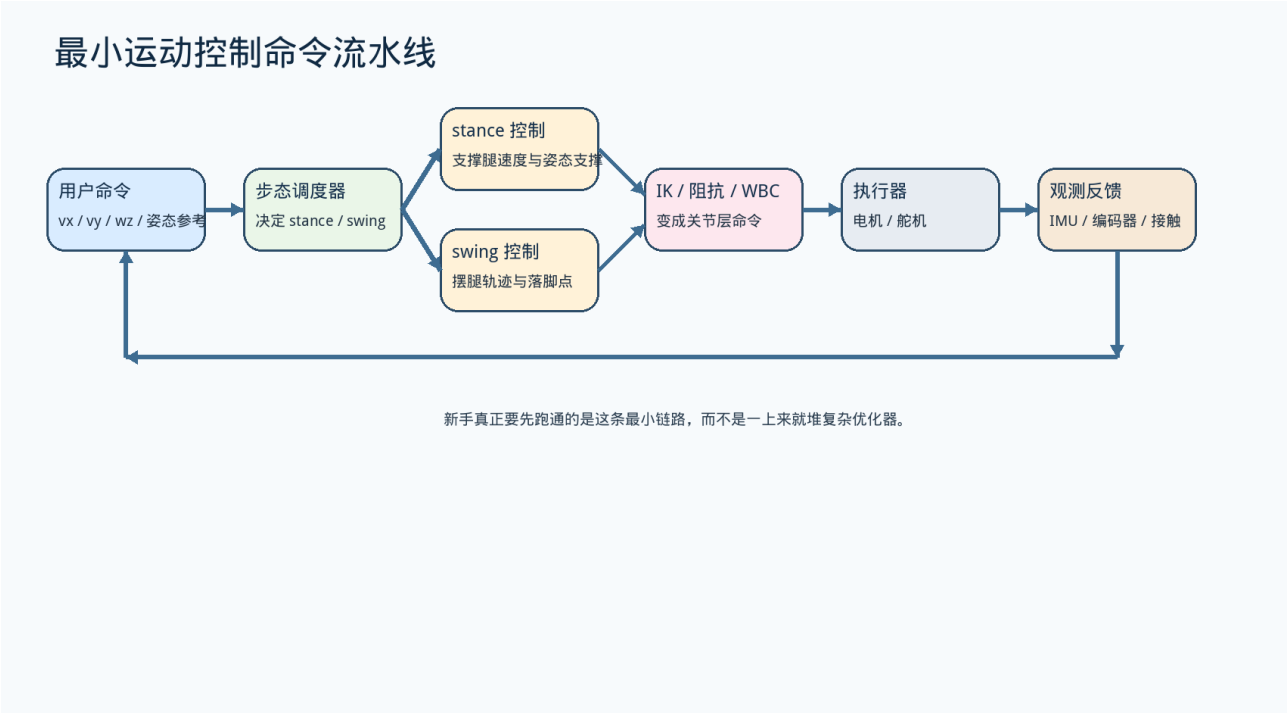
- 进入阈值：决定何时升步态
- 退出阈值：决定何时降步态
- 迟滞宽度：决定是否会来回抖动
- 渐变时长：决定切换是否生硬

附录 J：答辩 / 面试 / 评审常问问题

- 你的控制链路是如何分层的？
- 你为什么选择 trot 作为第一条动态步态？
- 你如何定义和检测步态切换成功？
- 你的状态估计依赖哪些传感器？
- 你如何做落脚点前推？
- 你如何避免速度命令突变导致失稳？
- 你的姿态控制和速度控制如何耦合？
- 你为什么在某个速度区间选择某个步态？
- 你如何做切换迟滞和过渡态？
- 你如何从 MuJoCo 迁移到 ROS2/Gazebo 或实机？

附录 K：最小控制器伪代码

如果你想真正理解“从零开始怎么写一个最小四足控制器”，下面这段伪代码值得反复看。



最小运动控制命令流水线

```
loop(dt):
  observation = read_sensors()

  posture_ref = user_posture_command()
  velocity_ref = user_velocity_command()

  gait_state = gait_scheduler.update(dt, velocity_ref)

  for leg in legs:
    if gait_state[leg] == swing:
      foot_target[leg] = swing_trajectory(leg, dt, velocity_ref)
    else:
```

```
        foot_target[leg] = stance_controller(
            leg,
            dt,
            velocity_ref,
            posture_ref,
            observation,
        )

    posture_compensation(foot_target, posture_ref, observation.imu)

    for leg in legs:
        joint_target[leg] = inverse_kinematics(foot_target[leg])
        joint_target[leg] = apply_joint_limits(joint_target[leg])

    send_joint_targets(joint_target)
    log_everything()
```

这段伪代码的意义不在于“足够高级”，而在于它把最小控制链路讲清楚了。

附录 L：推导前必须会的最小数学准备

这一附录不是要把你变成数学家，而是帮你补足“能读懂控制公式”的最小前置。

L.1 三角函数在四足控制里到底用来干什么

在四足机器人里，三角函数最常出现的场景有三个：

- 由足端目标距离推关节角
- 由机身姿态变化推足端位置修正
- 由 yaw 和速度推机身系 / 世界系转换

最需要记住的不是一堆恒等式，而是：

- `atan2(y, x)`：给你一个向量的方向角
- `cos(theta)` / `sin(theta)`：把一个长度投影到不同坐标轴
- 余弦定理：二连杆 IK 的核心

L.2 二维旋转矩阵的物理直觉

如果一个点在平面内绕原点旋转 `theta`，新的坐标可以写成：

```
[x']   [ cos(theta)  -sin(theta) ] [x]
[z'] = [ sin(theta)   cos(theta) ] [z]
```

它的物理含义是：

- 原先沿 `x` 轴的分量，一部分还留在 `x`，另一部分转到了 `z`
- 原先沿 `z` 轴的分量，也会有一部分转到 `x`

姿态补偿里为什么总会看到旋转矩阵？

因为你在做的事情，本质上就是：

- 机身姿态变了
- 默认站姿下的足端点也要跟着做旋转映射

L.3 三维姿态为什么总是先讲 roll / pitch / yaw

因为对很多四足平台来说：

- `roll`：决定左右高低
- `pitch`：决定前后高低
- `yaw`：决定航向和转身

在工程上，最常见并且最够用的办法是：

- 先把 roll / pitch / yaw 当作依次作用的小旋转
- 用旋转矩阵或近似方法作用到默认足端位置

L.4 小角度近似什么时候有用

当姿态变化比较小时：

```
sin(theta) ≈ theta
cos(theta) ≈ 1
```

这有什么用？

- 让你快速理解姿态变化与足端修正为什么近似线性
- 帮你看懂一些控制器里出现的“线性补偿项”

但要注意：

- 大姿态变化时不要滥用这个近似

L.5 为什么控制器里总有低通和斜坡

因为真实控制系统不喜欢突变。

最简单的一阶低通可以写成：

```
y[k] = alpha * y[k-1] + (1 - alpha) * x[k]
```

它的作用是：

- 把噪声抹平
- 但也会带来延迟

最简单的速度斜坡可以写成：

```
v_cmd[k] = clamp(v_ref[k], v_cmd[k-1] - a_max*dt, v_cmd[k-1] + a_max*dt)
```

它的作用是：

- 不让指令一下跳太大
- 避免控制器来不及跟

附录 M：单腿逆运动学从直觉到公式

这一节只推导“ 最值得你真正搞懂 ”的那一部分：二维二连杆 `IK`。

M.1 先把问题缩成二维

假设你已经把外展自由度单独处理掉了，现在只看髋膝平面：

- 大腿长度：`l1`
- 小腿长度：`l2`
- 足端目标：`(x, z)`

那么髋关节到足端的距离为：

```
d = sqrt(x^2 + z^2)
```

如果：

```
d > l1 + l2
```

说明足端超出最大可达域，`IK` 无解。

M.2 先求膝关节角

把大腿、小腿和髋到足端距离看成一个三角形，根据余弦定理：

```
d^2 = l1^2 + l2^2 - 2*l1*l2*cos(pi - knee)
```

整理后常见写法是：

```
cos(knee) = (l1^2 + l2^2 - d^2) / (2*l1*l2)
```

于是：

```
knee = arccos( clamp(cos(knee), -1, 1) )
```

为什么要 `clamp`？

因为数值误差会让本应是 `1.000000` 的值变成 `1.000001`，直接导致 `arccos` 崩掉。

M.3 再求髋关节角

从髋到足端目标点的方向角是：

```
phi = atan2(z, x)
```

而三角形内部还有一个辅助角：

```
beta = arccos( (l1^2 + d^2 - l2^2) / (2*l1*d) )
```

那么髋关节角通常可以写成：

```
hip = phi - beta
```

这里的符号可能因你的腿定义不同而不同，但物理意义不变：

- 先朝向足端方向
- 再减去或加上三角形内部几何补偿

M.4 为什么会出现“两支解”

因为几何上通常存在两种姿态：

- 膝向前折
- 膝向后折

在四足机器人里，你几乎总是只要其中一支。

所以工程上必须明确：

- 你保留的是哪支解
- 它与真实机械结构是否一致

M.5 从二维到三维怎么办

三维腿通常做法是：

- 先用 `y` 方向求外展角
- 再把三维问题投影到髋膝平面
- 再做上面的二维二连杆推导

也就是说，三维 IK 往往不是“完全重新推一遍”，而是：

```
3D -> 外展分解 -> 2D 二连杆
```

M.6 写进代码时最该防的 bug

- 足端超可达域
- `acos` 输入超 `[-1, 1]`
- 左右腿镜像符号反了
- `atan2` 传参顺序错了
- 关节零位和机械安装方向没统一

M.7 最小 IK 伪代码

```
d = sqrt(x*x + z*z)
d = clamp(d, d_min, d_max)

cos_knee = (l1*l1 + l2*l2 - d*d) / (2*l1*l2)
cos_knee = clamp(cos_knee, -1.0, 1.0)
knee = acos(cos_knee)
phi = atan2(z, x)
```

```
cos_beta = (l1*l1 + d*d - l2*l2) / (2*l1*d)
cos_beta = clamp(cos_beta, -1.0, 1.0)
beta = acos(cos_beta)

hip = phi - beta
```

附录 N：姿态补偿、步态相位与落脚点公式

N.1 默认站姿为什么是整个系统的基准

你可以把默认站姿理解成：

- 四条腿在“无速度、无姿态偏置”时的基准足端点

记作：

```
p_foot_nominal[leg]
```

那么真正控制时的足端目标往往是：

```
p_foot_target
= p_foot_nominal
+ posture_compensation
+ gait_offset
+ velocity_related_offset
```

这条式子很重要，因为它说明：

- 姿态、步态、速度不是互相替代
- 而是在足端层做叠加和协调

N.2 机身姿态补偿的最小表达

如果默认足端点在机身系中是 `p0`，姿态旋转矩阵是 `R_body`，那么一个经典做法是：

```
p_target = R_body * p0
```

如果你还想保留“原站立高度基准”，也常见写法：

```
p_target = R_body * p0 + p_body_translation
```

其中：

- `R_body` 体现 roll / pitch / yaw
- `p_body_translation` 体现高度、前后、左右平移修正

N.3 步态 phase 怎么变成 stance / swing

记全局相位为：

```
phase_global in [0, 1)
```

某条腿的局部相位为：

$$phase_leg = (phase_global + phase_offset[leg]) \bmod 1.0$$

如果：

$$phase_leg < duty_factor$$

则该腿处于 `stance`，否则处于 `swing`。

这条式子就是最小 gait scheduler 的数学核心。

N.4 为什么支撑足速度近似等于机身速度取反

在机身系里，如果身体向前走，而脚在地上不滑，那么从身体视角看：

- 地面像是在往后退
- 支撑脚相对身体也像在向后运动

所以最简单近似是：

$$v_foot_stance \approx -v_body$$

如果再考虑绕机身中心旋转：

$$v_foot_stance \approx -v_body - omega \times r_foot$$

其中：

- `omega` 是机身角速度
- `r_foot` 是脚相对机身中心的位置

N.5 简化落脚点为什么会有 `0.5*T_stance*v_cmd`

直觉是：

- 一次完整支撑相里，身体会在命令速度下往前走一段距离
- 如果想让下一步脚落下时正好支撑住身体，就要提前把脚往前放

为什么是 `0.5`？

因为一种最常见的粗略假设是：

- 当前摆腿落地后，会在接下来半个支撑周期内承担主要支撑作用

于是有：

$$p_touchdown \approx p_nominal + 0.5 * T_stance * v_cmd$$

这其实就是很多四足系统里常说的简化 `Raibert` 前推直觉。

N.6 为什么还要加反馈修正项

因为如果只用命令速度，你默认机器人“完全跟上了命令”。

但现实里经常是：

- 命令说前进 `0.4 m/s`
- 实际只跑到 `0.2 m/s`

所以一个更实用的着地点会加速度误差反馈：

```
p_touchdown
= p_nominal
+ 0.5 * T_stance * v_cmd
+ k_v * (v_actual - v_cmd)
```

这能帮助机器人：

- 跑慢了时更积极前探
- 跑快了时减少过冲

附录 O：最小状态估计公式

O.1 为什么新手先学互补滤波就够了

因为它足够简单，而且足够说明问题。

设：

- `gyro_angle`：陀螺仪积分得到的姿态
- `acc_angle`：由加速度估计的姿态

一个经典互补滤波是：

```
angle[k] = alpha * ( angle[k-1] + gyro_rate[k] * dt )
       + (1 - alpha) * acc_angle[k]
```

它的直觉是：

- 高频变化相信陀螺仪
- 低频漂移用加速度纠正

O.2 接触状态为什么能帮助状态估计

如果某条腿明确处于可靠支撑相，那么：

- 它相对地面的滑移可以近似更小
- 该足端位置可作为修正身体运动的参考

因此很多腿式机器人估计器都把接触状态当作一个“可信度门控”：

```
if contact_confidence[leg] > threshold:
   use foot constraint
else:
   ignore foot constraint
```

O.3 最小速度估计直觉

如果足端稳定接地，而你知道关节角和机身姿态，那么从腿运动学反推，可以得到一个粗略机身运动估计。

这不是最精确的方法，但非常适合建立直觉：

- 关节变化 + 接触脚约束
- 可以反推出身体相对脚的运动

O.4 为什么滤波参数不可能“一把梭”

如果 `alpha` 太大：

- 估计更平滑
- 但漂移和延迟会增大

如果 `alpha` 太小：

- 响应更快
- 但噪声会更大

这就是为什么状态估计调参必须和控制器一起看：

- 估计太慢，控制器会“后知后觉”
- 估计太抖，控制器会“跟着噪声乱动”

附录 P：接触力 QP 从 wrench 到足端力

P.1 为什么先单独学 contact QP

因为这是理解 WBC 最好的第一步。

如果你一上来就看 `x = [qdd, f]`，初学者很容易被一堆符号淹没；但如果先只看接触力分配，问题会清楚很多：

- 机器人现在有哪些腿在支撑？
- 机身此刻需要多大的合力和合力矩？
- 这些力怎么在几条支撑腿之间分？

只要把这三件事想明白，你就已经理解了 WBC 里最关键的“接触力”部分。

P.2 wrench_map 怎么把各腿接触力合成机身 wrench

对单条接触腿来说，接触力 `f_i` 对机身有两部分贡献：

```
力：  f_i
力矩： r_i x f_i
```

其中 `r_i` 是“该足端相对质心的位置向量”。

于是多条腿一起作用时：

```
w = sum_i [ f_i
        r_i x f_i ]
```

把它写成矩阵形式，就是：

```
w = A f
```

这就是 `wrench_map` 的本质。

在本仓库 `contact_qp.py` 和 `whole_body_qp.py` 里，你会看到每条腿都对应一块 6x3 的子矩阵：

```
[ I ]
[ [r_i]x ]
```

上 3 行负责合力，下 3 行负责合力矩。

P.3 摩擦锥、法向单边约束和投影

真实接触力不能想怎么取就怎么取，它至少要满足：

```
fz_i >= 0
sqrt(fx_i^2 + fy_i^2) <= mu * fz_i
```

这表示：

- 地面只能推脚，不能把脚往下吸
- 切向摩擦力不能超过法向支撑力乘摩擦系数

工程里经常会把圆锥近似成盒约束：

```
|fx_i| <= mu * fz_i
|fy_i| <= mu * fz_i
```

这样求解更简单，投影也更直接。

本仓库 `project_contact_force()` 做的正是这件事：

- 先夹住 `fz`
- 再根据 `mu * fz` 限制 `fx` 和 `fy`

所以它不是一个“后处理小技巧”，而是把数值结果强制拉回物理可行域的重要步骤。

P.4 anchor 与 share 两类正则项

只靠 `A f ~= w_des` 还不够，因为解通常不唯一。

比如总竖直力一样时：

- 可以四条腿平均分
- 也可以一条腿承担大部分，另外几条腿只轻轻碰地

为了让解更稳定、更符合工程预期，常加两类正则：

第一类，anchor：

$$\| f - f_{anchor} \|^2$$

意思是：接触力别离“锚点保持稳定时的期望力”太远。

第二类，share：

$$\| S_z f - f_{z,share} \|^2$$

意思是：竖直载荷按一定比例分给各条支撑腿，别让某条腿突然吃满载荷。

在本仓库里，对应的就是：

- `anchor_targets`
- `normal_share_targets`

P.5 从足端力到关节力矩时为什么会有符号差异

一个基本公式是：

$$\tau_{contact} = J_c^T f$$

但你在不同资料里，经常会看到：

$$\tau_{contact} = - J_c^T f$$

这不是谁对谁错，而是力的定义方向不同：

- 如果 `f` 定义为“控制器希望脚对地施加的力”，一个符号
- 如果 `f` 定义为“地面对脚的反作用力”，另一个符号

所以真正要看的是：

- Jacobian 的速度定义是什么
- 接触力向量的方向约定是什么

本仓库里两个位置值得对着看：

- `contact_qp.py` 中，单独把接触力映射成每条腿的关节力矩贡献
- `whole_body_qp.py` 中，最终恢复总力矩时使用

$$\tau = M \ddot{q} + h - J_c^T f$$

这恰好提醒你：看公式时一定要先看符号约定，不能只盯着表面长相。

附录 Q：Whole-Body QP 从任务到关节力矩

Q.1 为什么把 qdd 和 f 放进同一个变量

因为对腿式机器人来说，下面两件事本来就是耦合的：

- 关节要怎么加速

- 地面对每条支撑腿要提供多大反力

如果只解 `qdd` 不解 `f`，那接触会变成一个模糊的外界干扰；如果只解 `f` 不解 `qdd`，那摆腿和姿态任务又不好统一进去。

所以把它们拼成：

```
x = [qdd^T, f^T]^T
```

是很自然的选择。

Q.2 各任务怎样写成 J qdd ~= a*

很多 whole-body QP 的核心套路都是：

- 先在任务空间定义一个期望加速度
- 再让 Jacobian 乘 `qdd` 去逼近这个期望值

比如姿态或关节 posture：

```
a_post* = Kp (q* - q) + Kd (dq* - dq)
```

摆腿 swing：

```
a_swing* = Kp (p* - p) + Kd (v* - v)
```

支撑腿 stance / anchor：

```
a_stance* = Kp (p_anchor - p) + Kd (0 - v)
```

然后统一写成：

```
J qdd ~= a*
```

本仓库 `whole_body_qp.py` 里可以直接看到这种结构：

- posture：单位矩阵乘 `qdd`
- swing：足端 Jacobian 乘 `qdd`
- stance：足端 Jacobian 乘 `qdd`

Q.3 为什么 torque consistency 可以写成正则项

完整刚体动力学常写成：

```
M qdd + h = S^T tau + J_c^T f
```

严格做法可以把它作为等式约束直接塞进 QP。

但工程里还有另一条很实用的路：把它写成“尽量满足”的代价项，或者在解出 `qdd` 和 `f` 之后再重建 `tau`。

这样做的好处是：

- 实现更简单

- 数值更稳
- 更容易和已有任务式控制器混合

本仓库现在采用的是这种更工程化的写法：

- 把 `M qdd + h - J_c^T f` 放进正则项
- 解出来以后再恢复关节力矩

Q.4 解出来以后怎么恢复 tau

当解向量是：

```
x = [qdd, f]
```

那么恢复力矩最常见的形式就是：

```
qdd = x[0:nq]
f   = x[nq:]
tau = M qdd + h - J_c^T f
```

这里的直觉是：

- `M qdd + h`：如果没有接触，关节需要独自承担的动力学代价
- `J_c^T f`：接触力已经帮你承担掉的一部分
- 两者相减后，剩下的才需要电机真正输出

Q.5 本仓库 whole_body_qp.py 的对照阅读

如果你现在回去看 `mujoco_sim/wbc/whole_body_qp.py`，建议按这个顺序：

1. 先看 `solve_whole_body_tasks()`

- 它决定了变量维度、活跃接触腿、`contact_jacobian_t`
- 也决定了 `wrench_map`、`contact_force_targets`、`share_targets`

2. 再看 `accumulate_quadratic_term()`

- 它本质上就是把每个任务转成 Hessian 和 linear term 的累加

3. 再看 `solve_whole_body_qp()`

- 它不是黑箱商业求解器
- 而是一个带投影的迭代式求解过程

4. 最后看求解后的恢复部分

- `last_qdd_solution`
- `last_contact_forces`
- `tau = M qdd + h - J_c^T f`

如果你能把这四步对应回前面的公式，就说明 whole-body QP 已经真正入门了。

Q.6 调参顺序与排障抓手

WBC / QP 调参最怕的不是“参数多”，而是顺序错。

推荐顺序是：

1. 先确认接触可行域

- `mu`
- `fz_min / fz_max`
- 接触状态判断是否稳定

2. 再调任务权重

- posture
- swing
- stance
- base wrench

3. 再调正则

- torque regularization
- qdd regularization
- force regularization

4. 最后调数值稳定器

- warm start 保留比例
- step scale
- blend 范围和切换惩罚

最有用的排障日志往往不是“最后总力矩是多少”，而是：

- `wrench_map` 是否构造正确
- 每条腿当前是不是 active contact
- `contact_forces_world` 是否有明显跳变
- `whole_body_qp_blend` 是否在切换瞬间异常抖动
- `last_qdd_solution` 是否发散

附录 R：开源项目源码模式与参考代码骨架

这一附录的目标不是抄源码，而是把前面参考过的开源项目，压缩成几段真正能服务教程理解的“模式级代码骨架”。

下面这些代码块都做了两件事：

- 只保留最关键的控制结构

- 用教程自己的变量命名重写，不原样照搬外部仓库

所以你可以把它们理解成：

- 对外部源码的教学化总结
- 用来帮助你读真实仓库时快速定位关键结构

R.1 最小主循环骨架

参考来源：

- `Stanford Pupper Controller Description`
- `StanfordQuadruped`
- `Unitree Guide`
- 当前仓库 `mujoco_sim/controllers/`

典型主循环骨架通常都长这个样子：

```
while robot_is_running:
    sensors = read_sensors()
    state = estimate_state(sensors)

    user_cmd = read_user_or_nav_command()
    mode = update_mode_machine(state, user_cmd)

    gait_state = update_gait_scheduler(mode, state, user_cmd)
    foot_refs = build_foot_references(gait_state, state, user_cmd)

    actuator_cmd = low_level_control(foot_refs, state, mode)
    send_to_actuators(actuator_cmd)

    log_cycle(state, mode, gait_state, foot_refs, actuator_cmd)
```

你读外部源码时，如果找不到这条主线，就说明你还停留在“看见很多文件，但没看见控制流”。

R.2 gait scheduler 的最小骨架

参考来源：

- `Stanford Pupper`
- `CHAMP`
- `Unitree Guide`

典型 gait scheduler 的核心很简单：

```
global_phase = (time_now / gait_period) % 1.0

for leg in legs:
    leg_phase = (global_phase + phase_offset[leg]) % 1.0
    if leg_phase < duty_factor[leg]:
        leg_state[leg] = "stance"
    else:
        leg_state[leg] = "swing"
```

如果再工程化一点，通常还会带：

```
for leg in legs:
```

```
if leg_state_changed(leg):
    record_touchdown_or_liftoff_time(leg)
    start_transition_blend(leg)
```

你在真实项目里读 gait 代码时，优先找的不是“高级算法”，而是：

- `gait_period`
- `phase_offset`
- `duty_factor`
- `leg_state`

这四类量。

R.3 速度命令整形骨架

参考来源：

- `Stanford Pupper Operation Interface`
- `Unitree Guide`
- `CHAMP`

大多数开源项目最终都会把命令压到：

cmd = [vx, vy, wz]

再送进一个整形链：

```
cmd_raw = read_command()
cmd_deadband = apply_deadband(cmd_raw)
cmd_limited = apply_limits(cmd_deadband, max_vx, max_vy, max_wz)
cmd_ramped = apply_rate_limit(cmd_limited, accel_limits)
cmd_smooth = low_pass_filter(cmd_ramped)
```

然后才进入步态和足端层：

```
gait_mode = choose_gait_from_speed(cmd_smooth)
foot_plan = build_foothold_and_swing_targets(cmd_smooth, gait_mode, state)
```

这一骨架最大的教学意义是：

- 上层速度命令不是直接进电机
- 中间一定隔着命令整形和步态解释层

R.4 swing / stance 分发骨架

参考来源：

- `Stanford Pupper`
- `open-quadruped`
- `CHAMP`
- 当前仓库 `mujoco_sim/controllers/` 与 `mujoco_sim/wbc/command_builder.py`

最常见的腿级分发结构是：

```
for leg in legs:
    if leg_state[leg] == "swing":
        foot_target[leg] = swing_trajectory(
            lift_off_point[leg],
            touchdown_point[leg],
            swing_phase[leg],
        )
    else:
        foot_target[leg] = stance_reference(
            support_anchor[leg],
            desired_body_velocity,
            desired_body_attitude,
        )
```

更接近工程代码时，通常会变成：

```
if leg_state == "swing":
    target = swing_controller(...)
else:
    target = stance_controller(...)
```

这就是为什么本教程一直强调：

- swing 是轨迹问题
- stance 是支撑问题

如果一个仓库把这两条逻辑彻底混在一起，你读代码会很痛苦，调试也会很痛苦。

R.5 足端目标到关节命令的骨架

参考来源：

- `StanfordQuadruped`
- `open-quadruped`
- 本地 `refcode/py_apple_quadruped_robot/README.md`

只要项目不是直接做 torque-level whole-body 控制，通常都能看到类似结构：

```
for leg in legs:
    p_foot_des = body_pose_compensation(...) + gait_foot_target(...)
    q_leg_des = inverse_kinematics(leg, p_foot_des)
    write_joint_targets(leg, q_leg_des)
```

如果加上姿态闭环，一般只是前面多一层：

```
body_error = body_ref - body_state
body_comp = attitude_controller(body_error)
```

再把 `body_comp` 送进足端目标。

所以 IK 在工程里经常不是“整套控制器”，而是：

- 上层 body / gait 目标
- 到下层 joint 命令

之间的一座桥。

R.6 WBC 命令构造骨架

参考来源：

- 当前仓库 `mujoco_sim/wbc/command_builder.py`
- 当前仓库 `mujoco_sim/wbc/whole_body_qp.py`
- `WBC-quadruped-DOB`
- `TopHillRobotics/quadruped-robot`

当项目开始引入 WBC 时，常见结构不再是“只生成关节角”，而是先生成任务对象：

```
command = {
  posture_task,
  foot_tasks,
  contact_tasks,
  base_wrench_task,
}
```

然后才求解：

```
solution = solve_whole_body_tasks(robot_state, command)
tau = reconstruct_torque(solution)
```

这类源码的正确读法不是先盯 solver，而是先盯：

- task 是怎么组织的
- 哪些腿被当成 active contact
- 哪些目标是 posture / swing / stance / base wrench

只要这层看懂了，后面的 Hessian、linear term、projection 才会变得可读。

R.7 你在教程里应该怎样标注“参考代码段”

如果后续你继续扩教程，最推荐下面这种标注方式：

```
参考项目：Stanford Pupper / CHAMP / 当前仓库
参考目的：说明 gait scheduler 如何决定 stance / swing
建议关注：phase、duty_factor、leg_state、touchdown/liftoff
```

或者：

```
参考项目：当前仓库 mujoco_sim/wbc/
参考目的：说明 WBC 如何先构任务、再解 QP、最后恢复扭矩
建议关注：WbcCommand、wrench_map、solve_whole_body_tasks、last_qdd_solution
```

这样做的好处是：

- 读者知道这段代码为什么被引用
- 读者知道该抓哪个变量，而不是被整文件淹没
- 教程会更像“源码导读”，而不是“贴代码合集”

结语

四足机器人运动控制并不是一门“先学一个大而全算法就结束”的课程，而是一条层层搭建的工程路线。

真正的成长顺序通常是：

- 先会站
- 再会走
- 再会跑
- 再会切
- 再会稳
- 最后才是会优雅地做复杂控制

如果你按本教程的顺序推进，并坚持每一层都做到：

- 有理论解释
- 有日志可看
- 有测试可回归
- 有模式切换策略
- 有从仿真到实机的边界意识

那么你就不是“会调一个四足 demo”，而是在真正走向“会做四足机器人运动控制系统”。

参考资料

https://pupper.readthedocs.io/en/latest/reference/controller.html

https://github.com/unitreerobotics/unitree_guide

https://arxiv.org/abs/1712.02889

https://arxiv.org/abs/1909.06586

https://arxiv.org/abs/2208.00810

https://arxiv.org/abs/2410.09336

https://arxiv.org/abs/2205.01179

- Stanford Pupper Controller Description
- Unitree Guide
- Whole-Body Nonlinear Model Predictive Control Through Contacts for Quadrupeds
- Highly Dynamic Quadruped Locomotion via Whole-Body Impulse Control and Model Predictive Control
- A Whole-Body Controller Based on a Simplified Template for Rendering Impedances in Quadruped Manipulators
- A Novel Multi-Gait Strategy for Stable and Efficient Quadruped Robot Locomotion
- VAE-Loco: Versatile Quadruped Locomotion by Learning a Disentangled Gait Representation